
Documentação de Projeto

para o sistema

Stream Sentry

Versão 2.0

Projeto de sistema elaborado pelo aluno Rafael Parreira Chequer e apresentado ao curso de **Engenharia de Software** da **PUC Minas** como parte do Trabalho de Conclusão de Curso (TCC) sob orientação de conteúdo dos professores Danilo de Quadros Maia Filho, Leonardo Vilela Cardoso e Raphael Ramos Dias Costa, orientação acadêmica do professor Cleiton Silva Tavares e orientação de TCC II do professor Cleiton Silva Tavares.

14 de junho de 2026

Tabela de Conteúdo

Tabela de Conteúdo	ii
Histórico de Revisões	ii
1. Introdução	1
2. Modelos de Usuário e Requisitos	1
2.1 Descrição de Atores	1
2.2 Modelos de Usuários	2
2.3 Modelo de Casos de Uso e Histórias de Usuários	4
2.4 Diagrama de Sequência do Sistema e Contrato de Operações	6
3. Diagramas	10
3.1 Diagrama de Classes	10
3.2 Diagramas de Sequência	12
3.3 Diagramas de Comunicação	18
3.4 Arquitetura Lógica	22
3.5 Diagrama de Atividade	25
3.6 Diagrama de Componentes e Diagrama de Implantação	27
4. Projeto de Interface com Usuário	31
4.1 Interfaces Projetadas (Wireframes) — comuns a todos os atores	31
4.2 Interfaces Implementadas	38
5. Modelos de Projeto	46
5.1 Modelo de Dados	46
5.2 Glossário	47
6. Plano de Testes	49
6.1 Plano de Teste de Aceitação	49
6.2 Plano de Teste de Integração	51
7. Cronograma e Processo de Implementação	52
7.1 Cronograma	53
7.2 Metodologia de Desenvolvimento	56
7.3 Ferramentas e Pipeline de CI/CD	57
8. Post Mortem	58
8.1 Experiências Positivas	58
8.2 Experiências Negativas	59
8.3 Lições Aprendidas	59
8.4 Repositório do Trabalho	60

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Rafael Parreira Chequer	17/09/2025	Criação inicial do documento para entrega da A3, contendo diagrama de casos de uso, modelos de usuários e projeto de tela.	1.0
Rafael Parreira Chequer	13/10/2025	Criação do diagrama de sequência de sistema, diagrama de classes, diagrama de sequência e diagrama de comunicação e adição das demais interfaces.	1.1
Rafael Parreira Chequer	02/11/2025	Adição da arquitetura lógica, diagrama de atividades, componentes e implantação. Inclusão do modelo de dados e do glossário.	1.2
Rafael Parreira Chequer	18/11/2025	Adição do cronograma.	1.3
Rafael Parreira Chequer	30/11/2025	Adição dos planos de teste, metodologia de desenvolvimento, ferramentas e pipeline de CI/CD.	1.4
Rafael Parreira Chequer	14/12/2025	Finalizações gerais para última entrega de TCC 1.	1.5
Rafael Parreira Chequer	08/03/2026	Redefinição dos requisitos funcionais para maior clareza.	1.6
Rafael Parreira Chequer	29/03/2026	Atualização dos diagramas para conformidade com o novo sistema.	1.7
Rafael Parreira Chequer	26/04/2026	Atualização da formatação de diversas tabelas e de seus índices.	1.8

Nome	Data	Razões para Mudança	Versão
Rafael Parreira Chequer	24/05/2026	Atualização das APIs compatíveis na seção de post mortem.	1.9
Rafael Parreira Chequer	14/06/2026	Conformidade completa com o template e atualização de todos os pontos pendentes.	2.0

1. Introdução

Este documento apresenta a análise de usuário, contexto, interface e interação do sistema Stream Sentry, uma ferramenta web *open-source* (licença MIT) para automação de testes *end-to-end* em aplicações de videoconferência baseadas em *WebRTC*. Construída sobre uma interface em React, um servidor de orquestração em Go e um *runner* de automação em Node.js/Puppeteer, a ferramenta simula múltiplos usuários virtuais em salas de provedores como Jitsi Meet, Whereby e Zoom, coletando métricas de qualidade de experiência (QoE) em tempo real. A referência principal para a descrição do problema, domínio e requisitos é o Documento de Visão (anexo a este documento), que detalha o escopo, as funcionalidades, os usuários-alvo (desenvolvedores React, engenheiros de QA (Quality Assurance), equipes ágeis e pesquisadores acadêmicos) e as restrições do sistema.

2. Modelos de Usuário e Requisitos

Esta seção apresenta os modelos de usuário e os requisitos funcionais do sistema Stream Sentry, com base nas necessidades identificadas no Documento de Visão. São descritos os atores envolvidos, personas representativas, casos de uso, histórias de usuário, diagramas de interação e contratos de operações, garantindo alinhamento entre as expectativas dos usuários e as funcionalidades do sistema. O objetivo é fornecer uma base clara e estruturada para o desenvolvimento, priorizando usabilidade, automação e extensibilidade em cenários de teste de videoconferência.

2.1 Descrição de Atores

Os atores que interagem com o sistema Stream Sentry são definidos com base nos usuários-alvo descritos no Documento de Visão. Abaixo está a descrição de cada ator:

- **Desenvolvedor React:** Profissional que desenvolve aplicações de videoconferência em React, utilizando *WebRTC* ou plataformas como Jitsi Meet, Whereby e Zoom. Este ator utiliza o Stream Sentry para automatizar testes *end-to-end*, reduzindo o tempo e o esforço em validações manuais.
- **Engenheiro de QA:** Responsável por garantir a qualidade de aplicações de videoconferência, utilizando o sistema para executar testes automatizados e analisar relatórios detalhados com métricas de qualidade de experiência (QoE) como RTT, jitter, FPS, taxa de bits e taxa de falhas.
- **Equipe Ágil:** Grupo de profissionais (desenvolvedores, testadores e gerentes) que utilizam o sistema em fluxos de trabalho ágeis para integrar testes automatizados e relatórios (*JSON/CSV*) em seus processos.

2.2 Modelos de Usuários

Para representar os usuários do sistema, foram desenvolvidas personas que refletem as características, necessidades e objetivos dos atores identificados. As personas são baseadas nas necessidades levantadas no Documento de Visão e ajudam a guiar o design do sistema.

A Figura 1 apresenta a persona Lucas, Desenvolvedor React, que representa o ator Desenvolvedor React: um profissional focado em automatizar testes *end-to-end* de aplicações de videoconferência e reduzir o esforço com validações manuais.



Figura 1: Lucas Desenvolvedor React

A Figura 2 apresenta a persona Ana, Engenheira de QA, que representa o ator Engenheiro de QA: responsável por garantir a qualidade das aplicações e por analisar as métricas de qualidade de experiência (QoE) coletadas pelo sistema.



Figura 2: Ana Engenheira de QA

A Figura 3 apresenta a persona Felipe, Tech Lead, que representa o ator Equipe Ágil: responsável por integrar os testes automatizados e os relatórios exportados (*JSON/CSV*) ao fluxo de trabalho da equipe.



Figura 3: Felipe Tech Lead

2.3 Modelo de Casos de Uso e Histórias de Usuários

O diagrama de casos de uso representa as interações principais dos atores com as funcionalidades do Stream Sentry. A Figura 4 apresenta esse diagrama.

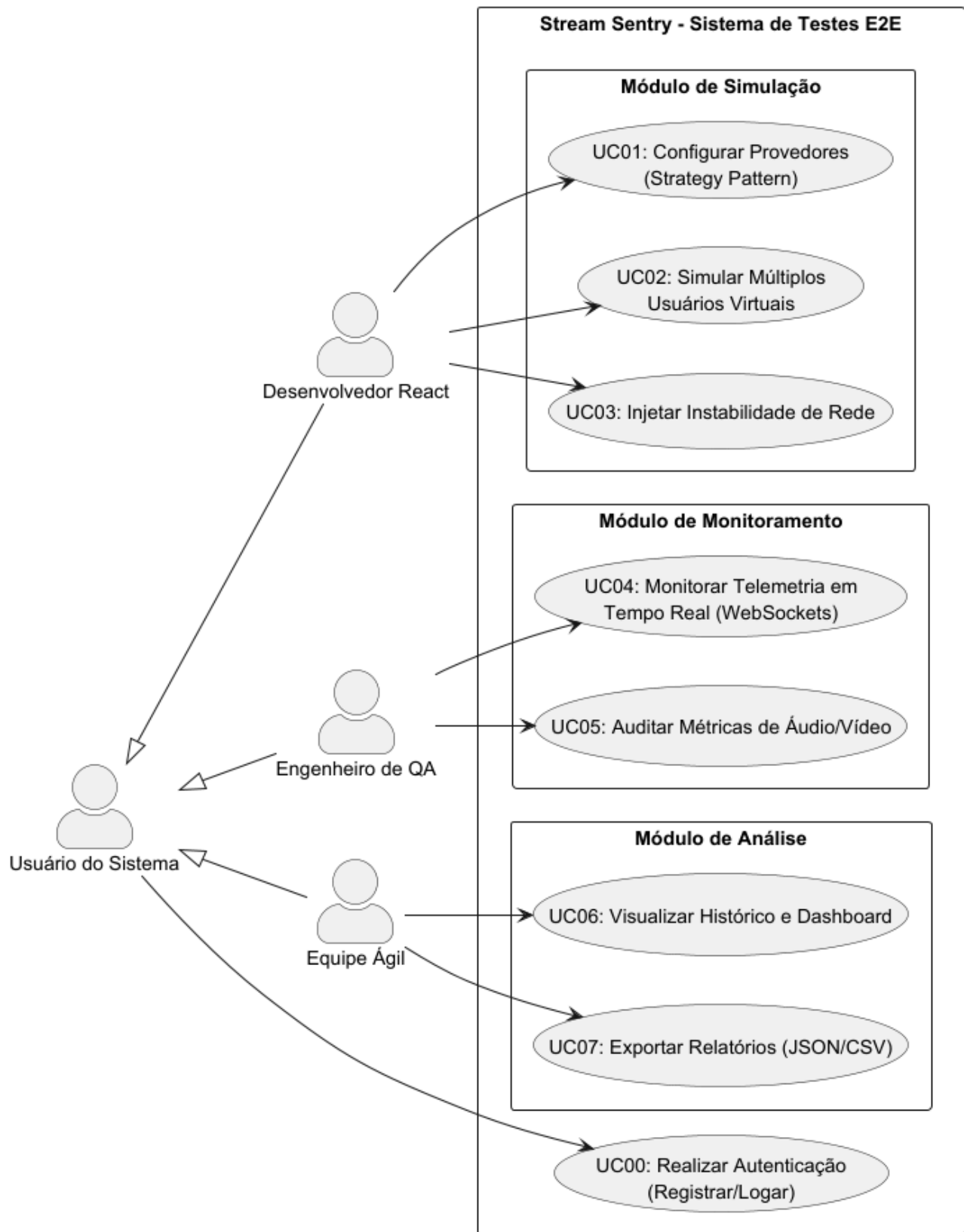


Figura 4: Diagrama de Casos de Uso do Stream Sentry

O diagrama ilustra os atores (Desenvolvedor React, Engenheiro de QA e Equipe Ágil) e suas interações com os casos de uso que sustentam o ecossistema do Stream Sentry. Cada interação reflete uma funcionalidade crítica voltada para a automação de simulações, o monitoramento de fluxos de mídia e a análise de dados para tomada de decisão.

Abaixo está a lista de histórias de usuário mapeadas, utilizando identificadores únicos para referência:

- **US00:** Como usuário do sistema, eu quero realizar o registro e login, para garantir que minhas configurações de teste e relatórios sejam armazenados de forma segura e privada.
- **US01:** Como Desenvolvedor React, eu quero configurar provedores de vídeo via *Strategy Pattern*, para garantir a compatibilidade com *WebRTC*, Jitsi, Whereby e Zoom sem acoplamento rígido.
- **US02:** Como Desenvolvedor React, eu quero simular múltiplos usuários virtuais simultâneos, para verificar o comportamento da aplicação sob carga.
- **US03:** Como Engenheiro de QA, eu quero monitorar a telemetria em tempo real via *WebSockets*, para identificar falhas de latência ou interrupções instantaneamente durante a execução.
- **US04:** Como Engenheiro de QA, eu quero auditar métricas de qualidade de experiência (QoE) de áudio e vídeo, como RTT, jitter e FPS, para validar a qualidade antes da entrega do software.
- **US05:** Como membro de uma Equipe Ágil, eu quero visualizar o histórico de testes no *dashboard*, para acompanhar a estabilidade do produto ao longo do ciclo de desenvolvimento.
- **US06:** Como membro de uma Equipe Ágil, eu quero exportar relatórios detalhados em *JSON* ou *CSV*, para compartilhar dados técnicos e integrar os resultados a ferramentas externas de análise.
- **US07:** Como Desenvolvedor React, eu quero injetar instabilidades de rede (perfis de chaos como 3G lenta, alta latência e offline) durante o teste, para verificar a resiliência da aplicação em cenários críticos.

2.4 Diagrama de Sequência do Sistema e Contrato de Operações

Esta seção descreve a dinâmica operacional do Stream Sentry, detalhando as interações entre os atores e o ecossistema do sistema. A modelagem foi atualizada para contemplar a arquitetura modular, a comunicação bidirecional via *WebSockets* para telemetria de alta fidelidade e os mecanismos de injeção de instabilidade de rede.

Nesta etapa, o foco reside na apresentação dos Diagramas de Sequência do Sistema (DSS). Um DSS é um artefato da UML (Unified Modeling Language) que ilustra, para um cenário específico de um caso de uso, os eventos gerados por atores externos, sua ordem e os eventos internos ao sistema. Diferente de um diagrama de sequência detalhado, o DSS trata o sistema como uma "caixa-preta",

omitindo a complexidade de classes ou componentes internos e focando estritamente no contrato de interface entre o usuário e a solução tecnológica.

O estágio inicial compreende a definição das premissas de carga e estabilidade. Através da interface de configuração, o Desenvolvedor React estipula o volume de usuários virtuais e o provedor de vídeo alvo. Neste momento, o sistema aplica o padrão *Strategy* para validar a compatibilidade da API (*WebRTC*, *Jitsi*, *Whereby* ou *Zoom*) e persiste o cenário no banco de dados, garantindo que o motor de execução receba uma configuração íntegra e pronta para o disparo. A Figura 5 apresenta o diagrama de sequência correspondente a essa etapa de configuração.

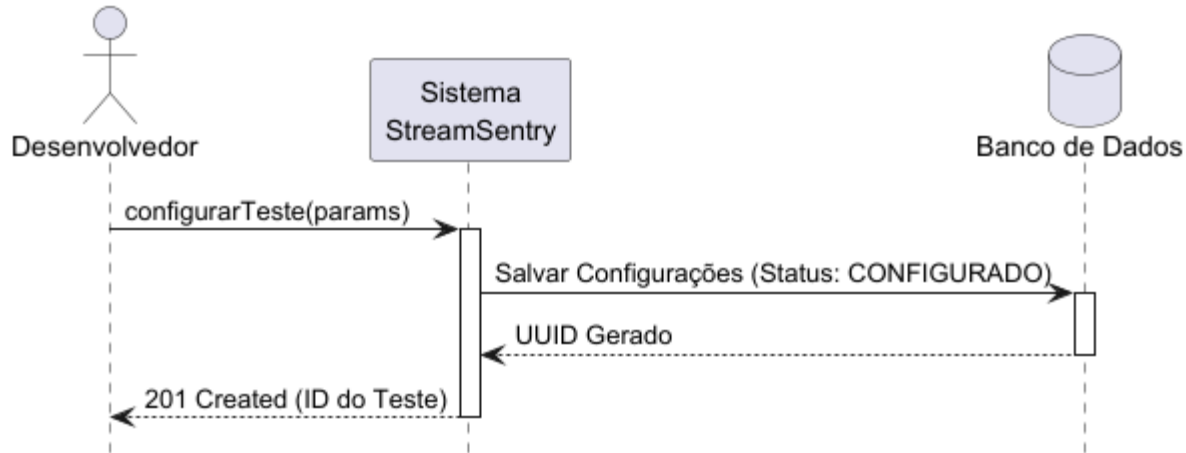


Figura 5: Diagrama de Sequência para Configuração de Cenário de Teste

A Tabela 1 formaliza a operação `configurarTeste`, ponto de entrada para a definição de cenários de carga. É indispensável que o Desenvolvedor React esteja autenticado e que o provedor de vídeo alvo seja validado pela *ProviderFactory*. Como pós-condição, o sistema assegura a integridade dos parâmetros — como volume de usuários e regras de instabilidade — persistindo-os no banco de dados sob um identificador de sessão e deixando o motor de execução em estado de prontidão imediata.

Contrato	Configurar Teste de Carga e Estabilidade
Operação	<code>configurarTeste(quantidadeUsuarios: int, api: string, duracao: int, instabilidade: JSON)</code>
Referências cruzadas	US01, US05, US06
Pré-condições	- Usuário está autenticado. - Provedor de vídeo validado pela <i>Factory</i>.
Pós-condições	- Parâmetros salvos no MongoDB - ID de sessão. - Status "CONFIGURADO"

Tabela 1: Contrato da Operação de Configuração

A fase de execução é caracterizada pela orquestração de *Workers Puppeteer* que simulam o comportamento humano em ambientes *headless*. Durante este ciclo, o sistema estabelece um canal de dados persistente que transmite métricas de performance (latência, *bitrate* e falhas) instantaneamente para o *Dashboard*. O diferencial deste estágio é a capacidade de intervenção do Engenheiro de QA, que pode acionar gatilhos de instabilidade para avaliar a resiliência da aplicação sob estresse de rede. A Figura 6 apresenta o diagrama de sequência dessa fase de execução e monitoramento.

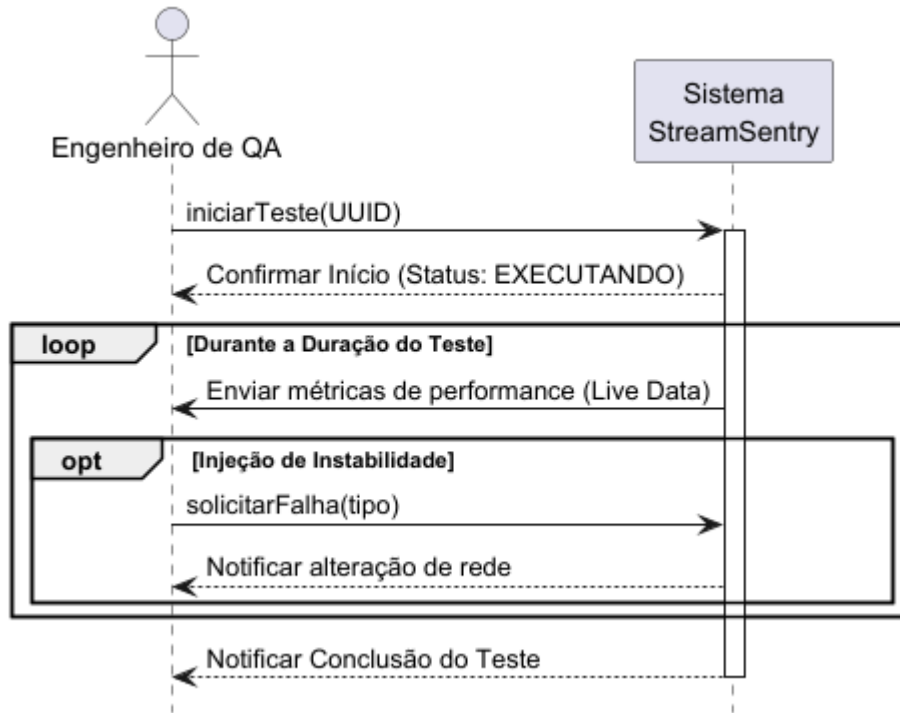


Figura 6: Diagrama de Sequência para Execução e Monitoramento

O contrato detalhado na Tabela 2 define o rigor da função executarTeste, responsável por disparar os *Workers Puppeteer*. As pré-condições exigem uma configuração válida e conexão ativa com a *API* externa. As pós-condições garantem que a simulação foi realizada conforme o planejado, com métricas de performance (latência e falhas) coletadas continuamente via *WebSockets*. Ao final, o sistema realiza o *teardown* das instâncias e atualiza o *status* do teste para "CONCLUÍDO".

Contrato	Executar Simulação e Telemetria em Tempo Real
Operação	executarTeste(idSessao: string)
Referências cruzadas	US02, US05
Pré-condições	- Teste configurado com ID de sessão válido. - Canal de <i>WebSocket</i> pronto.
Pós-condições	- Simulação executada. - Telemetria persistida. - Status "CONCLUÍDO".

Tabela 2: Contrato da Operação de Execução

O fechamento do fluxo ocorre com a transição dos dados da telemetria em tempo real para o processamento de dados históricos. O sistema agrega as séries temporais coletadas, gerando *insights* sobre a Qualidade de Experiência (QoE). Focado na *Developer Experience* (DX), esta etapa permite a exportação de relatórios estruturados (*JSON/CSV*), viabilizando a auditoria de resultados pela Equipe Ágil. A Figura 7 apresenta o diagrama de sequência dessa etapa de exportação de dados analíticos.

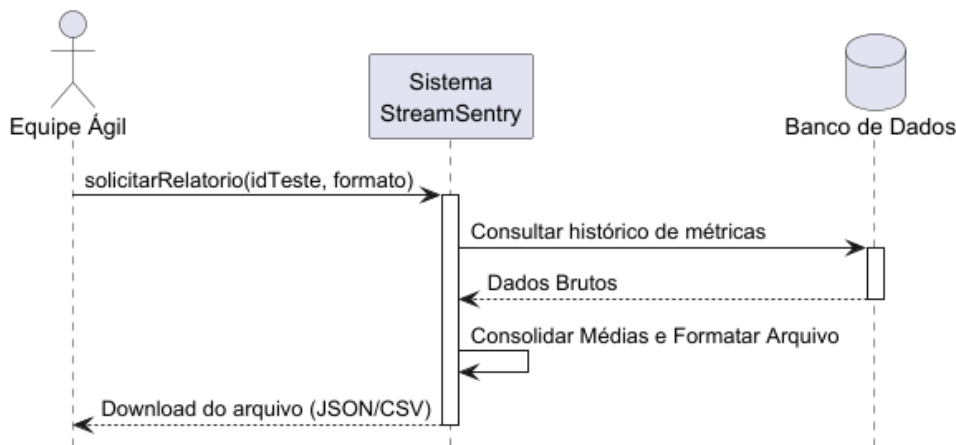


Figura 7: Diagrama de Sequência para Exportação de Dados Analíticos

A Tabela 3 estabelece as regras para a operação visualizarRelatorio, que governa a transição da telemetria em tempo real para o processamento histórico. O contrato exige que o teste tenha sido finalizado, permitindo que o sistema agregue as séries temporais para calcular a Qualidade de Experiência (QoE). O resultado é a entrega de um relatório técnico em formatos estruturados (*JSON/CSV*).

Contrato	Gerar e Exportar Relatório Analítico de Performance
Operação	visualizarRelatorio(idSessao: string, formato: string)
Referências cruzadas	US03, US06
Pré-condições	- Teste concluído. - Formato de saída (<i>JSON</i> ou <i>CSV</i>) suportado.
Pós-condições	- Dados agregados em relatório técnicos. - Arquivo disponibilizado para exportação.

Tabela 3: Contrato da Operação de Visualizar Relatório

Este fluxo consolida a porta de entrada do sistema, cobrindo desde a criação da conta até a obtenção do *token* de acesso necessário para as demais operações. A Figura 8 apresenta o diagrama de sequência de autenticação e registro, contemplando os dois cenários alternativos: o cadastro de novo usuário (com verificação de e-mail e persistência) e a realização de *login* (com validação de credenciais e geração do *token JWT*).

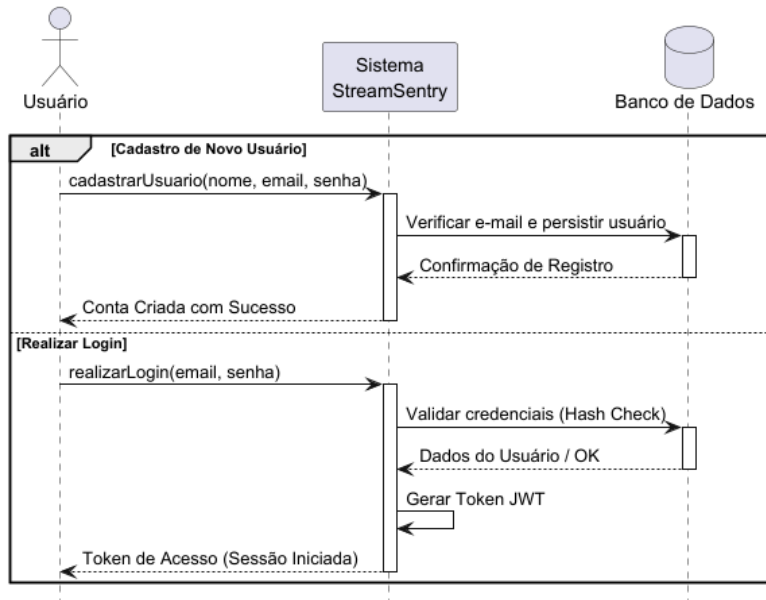


Figura 8: Diagrama de Sequência de Autenticação e Registro

A Tabela 4 formaliza as regras de acesso ao Stream Sentry, servindo como pré-requisito técnico para a execução de qualquer teste ou consulta de relatório.

Contrato	Realizar Cadastro e Autenticação
Operação	cadastrarUsuario(nome, email, senha) / realizarLogin(email, senha)
Referências cruzadas	US00
Pré-condições	- Para Cadastro: E-mail único no sistema. - Para Login: Credenciais válidas previamente registradas.
Pós-condições	- Usuário persistido com senha criptografada. - <i>Token</i> de acesso (<i>JWT</i>) gerado e devolvido ao <i>frontend</i>. - Estado do sistema alterado para "Autenticado", liberando acesso ao <i>Dashboard</i>.

Tabela 4: Contrato da Operação de Autenticação

3. Diagramas

A seção de Diagramas usa artefatos *UML* para apresentar uma visão integrada da estrutura, comportamento e arquitetura do Stream Sentry, cobrindo desde a modelagem estática até os fluxos dinâmicos e a implantação. Esses modelos sustentam a implementação, promovem a separação de responsabilidades e facilitam a compreensão, manutenção e escalabilidade do sistema em todos os seus níveis.

3.1 Diagrama de Classes

O Diagrama de Classes do Stream Sentry apresenta uma arquitetura organizada para suportar a simulação de múltiplos usuários e telemetria em tempo real, estruturada nos pacotes lógicos de Modelos (Domínio), *Core* (Lógica de Execução) e Integração & Estratégia. A classe *TestEngine* atua como o orquestrador central e núcleo do sistema, gerenciando uma lista de *WorkerNode*. Estes *WorkerNode* são unidades isoladas responsáveis por realizar o *spawn* de instâncias individuais, coletar estatísticas de *RTC* e executar a injeção de falhas (instabilidades) de rede de forma granular.

O padrão *Strategy* é formalizado pela interface *IConferenceProvider*, que define o contrato único para as implementações de *WebRTC*, *Whereby*, *Zoom* e *Jitsi*. Essa interface permite que a *ProviderFactory* instancie dinamicamente os *drivers* solicitados pelos *WorkerNode* sem gerar acoplamento rígido. Para garantir a atualização instantânea do *dashboard*, a classe *SocketManager* atua como um componente transversal, sendo utilizada pelo *TestEngine* para dados ao vivo e pelos *WorkerNode* para o envio de telemetria bruta (*raw*). Enquanto as entidades de domínio *Usuario*, *Teste* e *Relatorio* mantêm a lógica de *estado*, progresso e persistência no banco de dados , a

arquitetura garante a separação de responsabilidades ao permitir que o Relatório consolide as métricas finais para exportação em formatos estruturados.

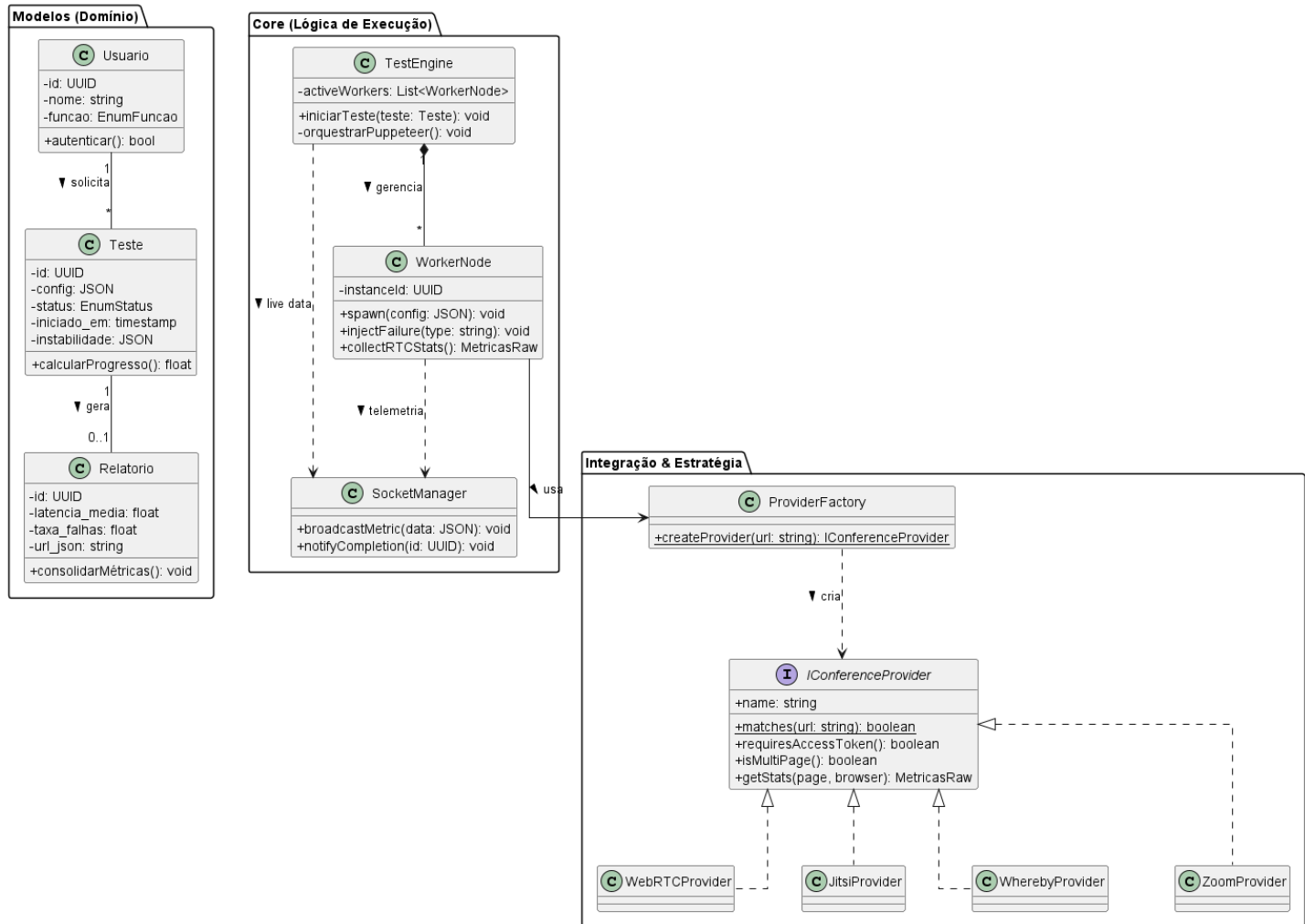


Figura 9: Diagrama de Classes do Stream Sentry

3.2 Diagramas de Sequência

Os diagramas apresentados nesta seção detalham a colaboração entre os componentes internos do Stream Sentry, revelando a lógica de orquestração e o processamento assíncrono que sustenta as funcionalidades do sistema.

A Figura 10 apresenta o primeiro diagrama detalhado, referente ao fluxo de Configuração e Validação, no qual a integridade dos dados é assegurada por meio do desacoplamento de provedores. O processo inicia-se na interface React (TypeScript), que encaminha os parâmetros ao *Core API* em Go. Ao receber a requisição, o sistema utiliza a *ProviderFactory* para instanciar o *driver* correspondente através da interface *IConferenceProvider*. Esta validação lógica garante que o

teste só avance se o *driver* da API solicitada (*WebRTC* genérico, Jitsi, Whereby ou Zoom) estiver disponível e operacional. Somente após essa confirmação, o registro é persistido no MongoDB com o status "CONFIGURADO" e um identificador único de sessão, garantindo a rastreabilidade necessária para as etapas seguintes.

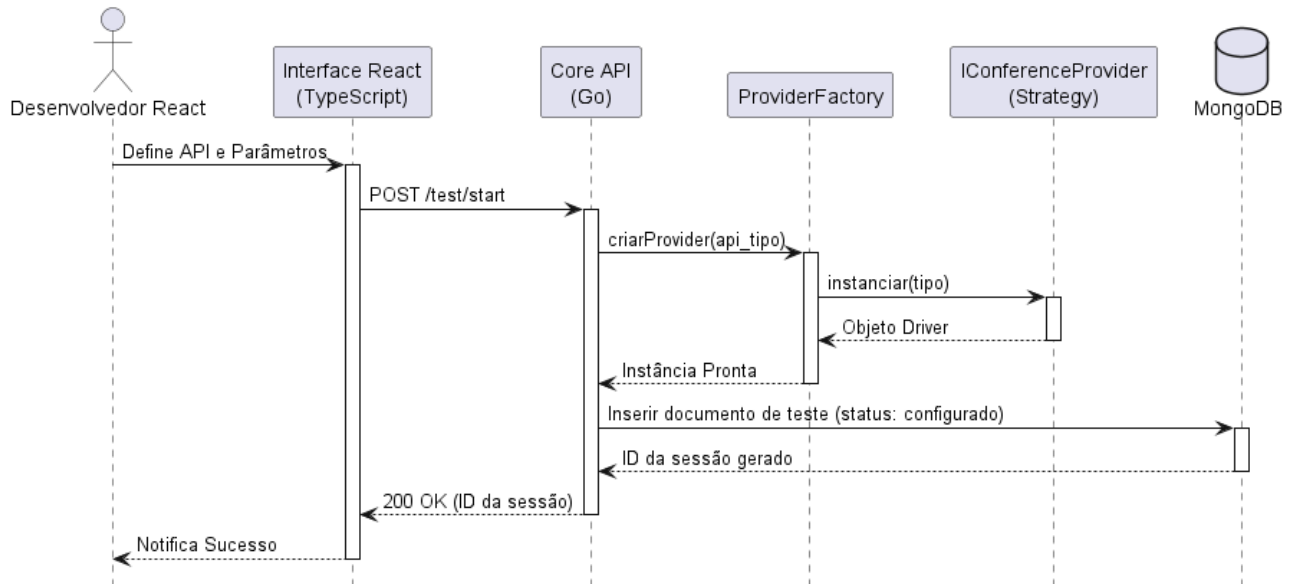


Figura 10: Diagrama de Sequência de Configuração e Validação

A Figura 11 apresenta o segundo fluxo, que ilustra a Execução em Tempo Real, evidenciando a arquitetura orientada a eventos necessária para a simulação de carga. Ao receber a requisição de início (POST /test/start), o *TestEngine* assume a coordenação do ciclo de vida da simulação, realizando o *spawn* de instâncias isoladas de *WorkerNode* via *Puppeteer*. Cada *worker* estabelece sua própria sessão de vídeo e entra em um loop de monitoramento contínuo. O diferencial reside na coleta de estatísticas *WebRTC* de baixo nível, que são encaminhadas ao *SocketManager* para difusão imediata (*broadcast*) à interface React. Simultaneamente, o *Core* persiste as séries temporais no MongoDB (coleção *telemetry_events*), permitindo que a visualização ao vivo ocorra sem interrupções ou bloqueios de processamento.

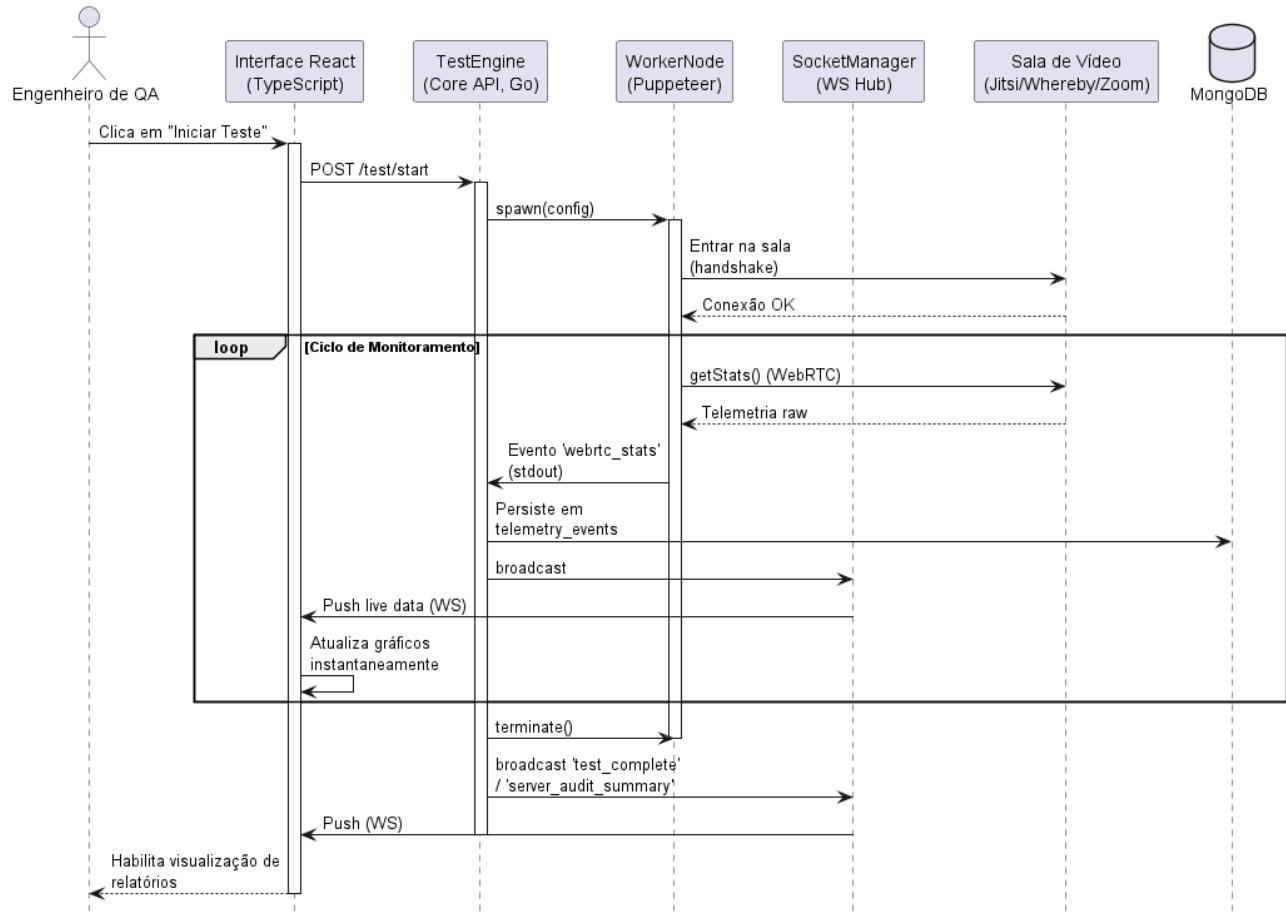


Figura 11: Diagrama de Sequência de Execução em Tempo Real

A Figura 12 apresenta, por fim, o fluxo de Geração de Relatórios, que demonstra a aplicação do princípio de Separação de Responsabilidades (SoC) no tratamento de dados históricos. Este processo é acionado sob demanda, isolando o processamento analítico do motor de execução principal. O *ReportGenerator* executa consultas de agregação no MongoDB para consolidar a telemetria armazenada (coleções *test_history* e *telemetry_events*), calculando métricas de Qualidade de Experiência (QoE), como RTT médio e taxa de falhas. Utilizando estratégias de formatação específicas, o componente converte o *dataset* processado em arquivos *JSON* ou *CSV* e os transmite via *stream* para o usuário. Essa abordagem garante que a extração de grandes volumes de dados não impacte a performance de outros testes que possam estar em execução simultânea.

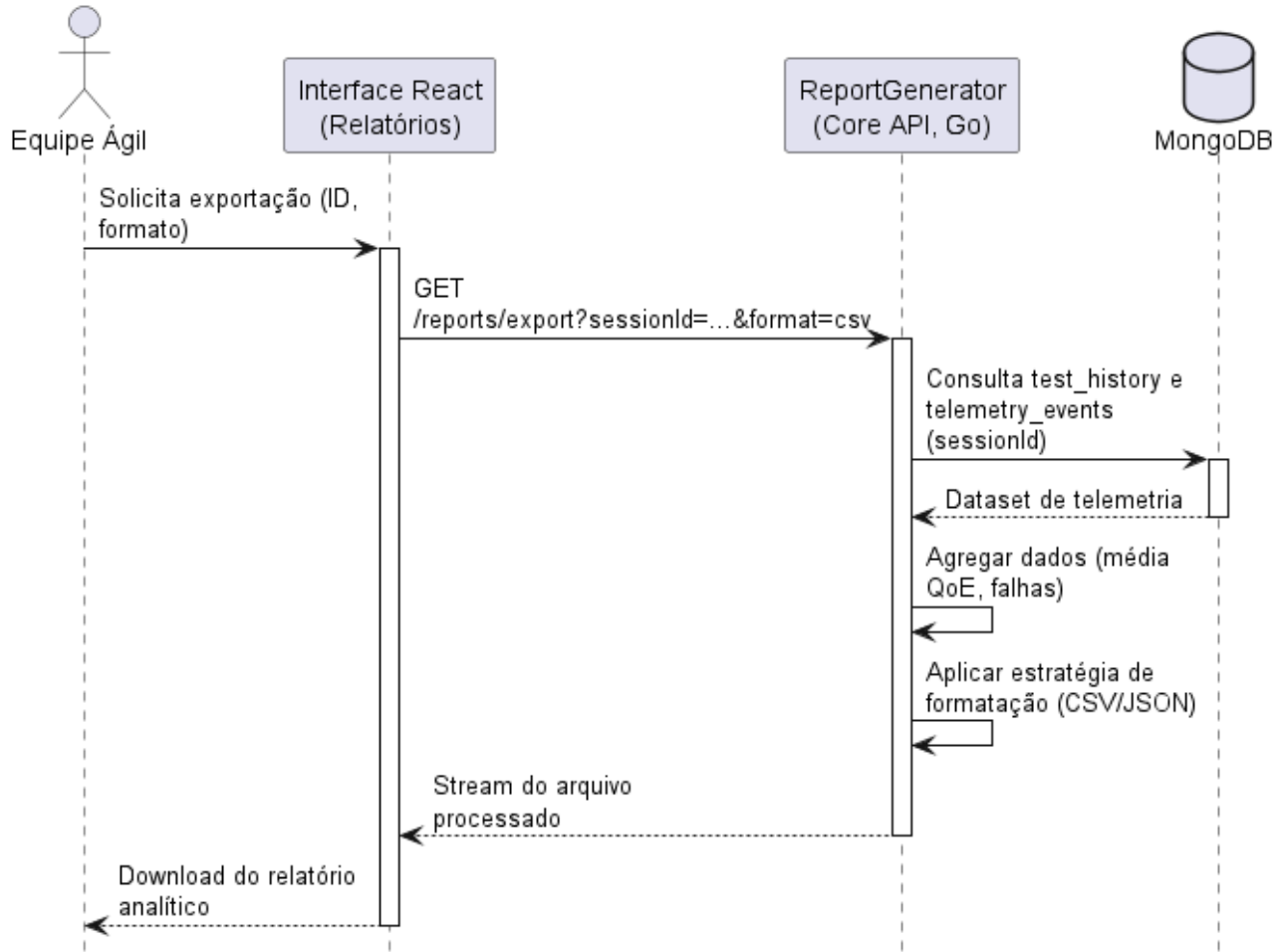


Figura 12: Diagrama de Sequência de Geração de Relatórios

Além dos fluxos anteriores, o sistema permite o controle do teste em tempo real. A Figura 13 apresenta o diagrama de sequência dessa operação: o Engenheiro de QA, pela interface, ajusta a concorrência (usuários virtuais) ou o perfil de chaos, ou ainda pausa, retoma e finaliza o teste. A interface envia a requisição ao authServer (POST /test/control, /test/pause, /test/resume ou /test/stop), que registra o novo estado no arquivo control.json via mergeWriteAuditControl(). O runner (run-audit.mjs) lê esse arquivo periodicamente (≈ 1 s) e aplica as mudanças — ajustando o pool de usuários, injetando instabilidade via CDP nos Chromium ou congelando a permanência na chamada. No caso de finalização, o authServer cancela o processo do runner e difunde o evento test_stopped pelo WebSocket.

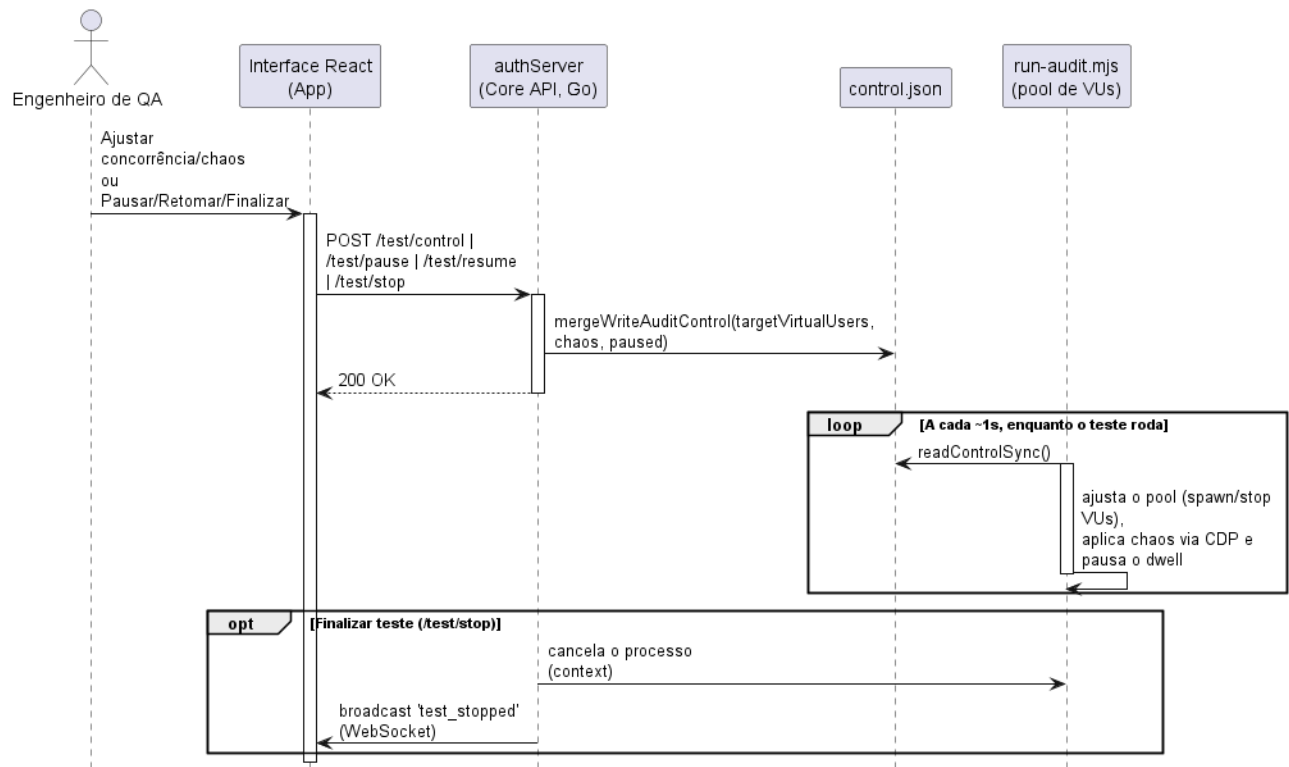


Figura 13: Diagrama de Sequência de Controle ao Vivo do Teste

A Figura 14 apresenta o diagrama de sequência da consulta ao histórico. Ao abrir a aba Histórico, a interface solicita GET /tests/history ao authServer, que lê a coleção test_history no MongoDB (dbLoadTestHistory()) e devolve a lista de sessões. Ao clicar em "Ver auditoria", a interface solicita GET /reports/session/{id}; o authServer recupera o registro e os eventos da sessão (dbFindTestHistoryRecord() e dbLoadSessionEvents(), sobre test_history e telemetry_events) e a interface reconstrói a auditoria a partir do log com buildTelemetrySnapshotFromEvents().

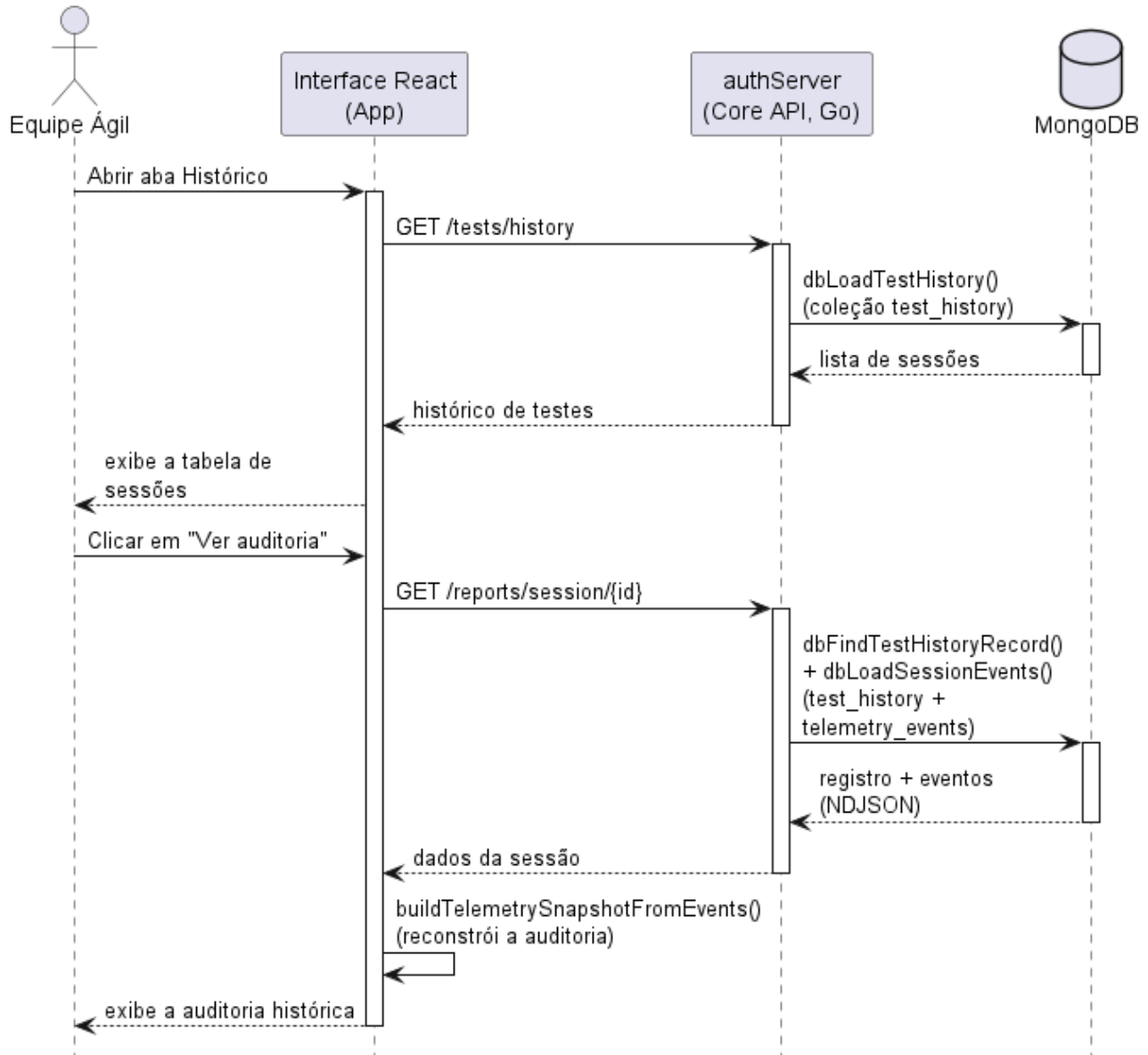


Figura 14: Diagrama de Sequência de Consulta ao Histórico

3.3 Diagramas de Comunicação

A Figura 15 apresenta o diagrama de comunicação de Configuração e Validação, que ilustra o estágio inicial de preparação do ambiente de teste, onde a flexibilidade arquitetural é o requisito principal. O fluxo demonstra a colaboração entre a interface App (App.tsx), o Core API em Go (authServer) e o *runner* (run-audit.mjs), evidenciando como o sistema desacopla a lógica de negócio das implementações específicas de videoconferência (*WebRTC* genérico, Jitsi, Whereby ou Zoom) por meio dos padrões *Factory* e *Strategy*. Após o envio da requisição (mensagem 2: *POST* /test/start, em handleTestStart), o authServer valida os parâmetros (mensagem 3: validateTechnicalConfig()) e dispara o *runner* (mensagem 4: runAuditTest()). No *runner*, a mensagem 5 seleciona em tempo de execução o provedor adequado à URL alvo, por meio da *ProviderFactory*, que instancia a implementação correspondente de *IConferenceProvider*. Essa estrutura assegura que cada cenário utilize o *driver* correto de forma desacoplada e que apenas configurações válidas avancem para a fase de execução.

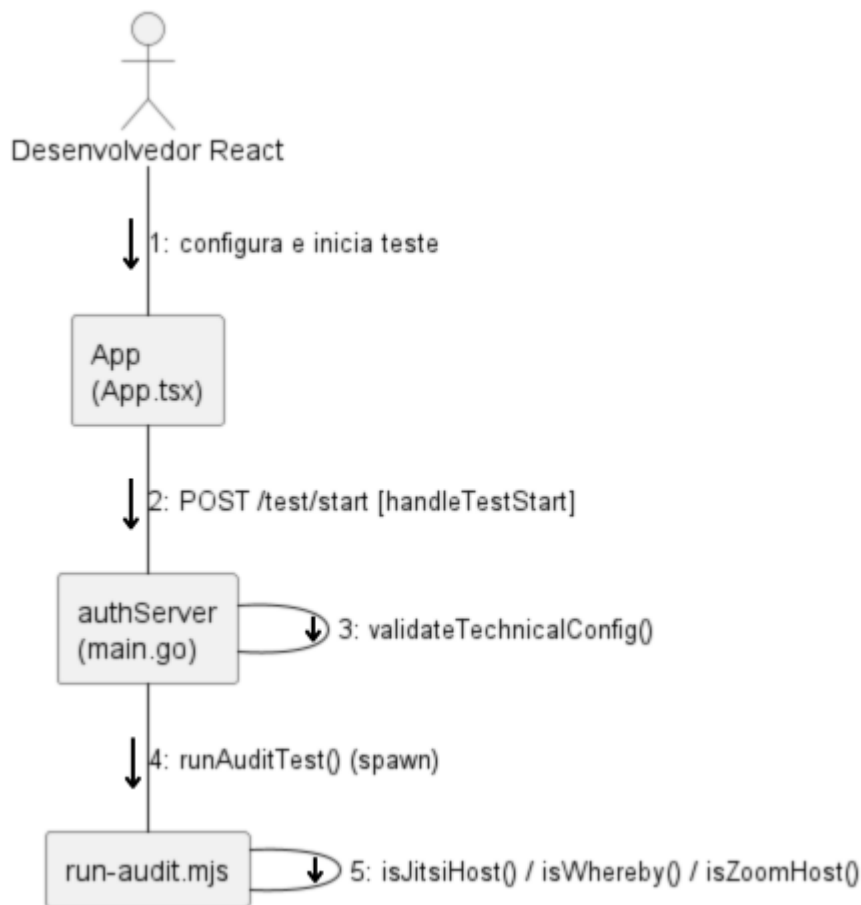


Figura 15: Diagrama de Comunicação de Configuração e Validação

A Figura 16 apresenta o diagrama de comunicação de Execução e Telemetria, que foca na dinâmica do "caminho quente" (*hot path*), onde a baixa latência é prioritária. Ele revela uma estrutura na qual o `authServer` (*Core API* em Go) atua como ponto central de coordenação entre a interface (App), o *runner* `run-audit.mjs` (função `runVirtualUser`, sobre Puppeteer), o *hub* de *WebSocket* (`telemetryHub`) e a persistência (MongoDB). A numeração das mensagens destaca o ciclo de vida assíncrono: após o disparo do teste (mensagem 2: `POST /test/start`) e o *spawn* do *runner* (mensagem 3: `runAuditTest()`), cada usuário virtual coleta as estatísticas *WebRTC* da sala (mensagem 4: `collectWebRTCSummary()` / `getStats()`) e encaminha cada amostra ao `authServer` pelo *stdout* (mensagem 5: evento `webrtc_stats`). O `authServer` persiste o evento na coleção `telemetry_events` do MongoDB (mensagem 6: `dbInsertTelemetryEvent()`) e, em paralelo, delega ao `telemetryHub` a difusão em tempo real (mensagem 7: `broadcast()`), de modo que o *hook* `useTelemetryWS` atualize a interface React instantaneamente (mensagem 8: `push` via `/ws/telemetry`). Esse desenho garante que a persistência não bloqueie o monitoramento ao vivo.

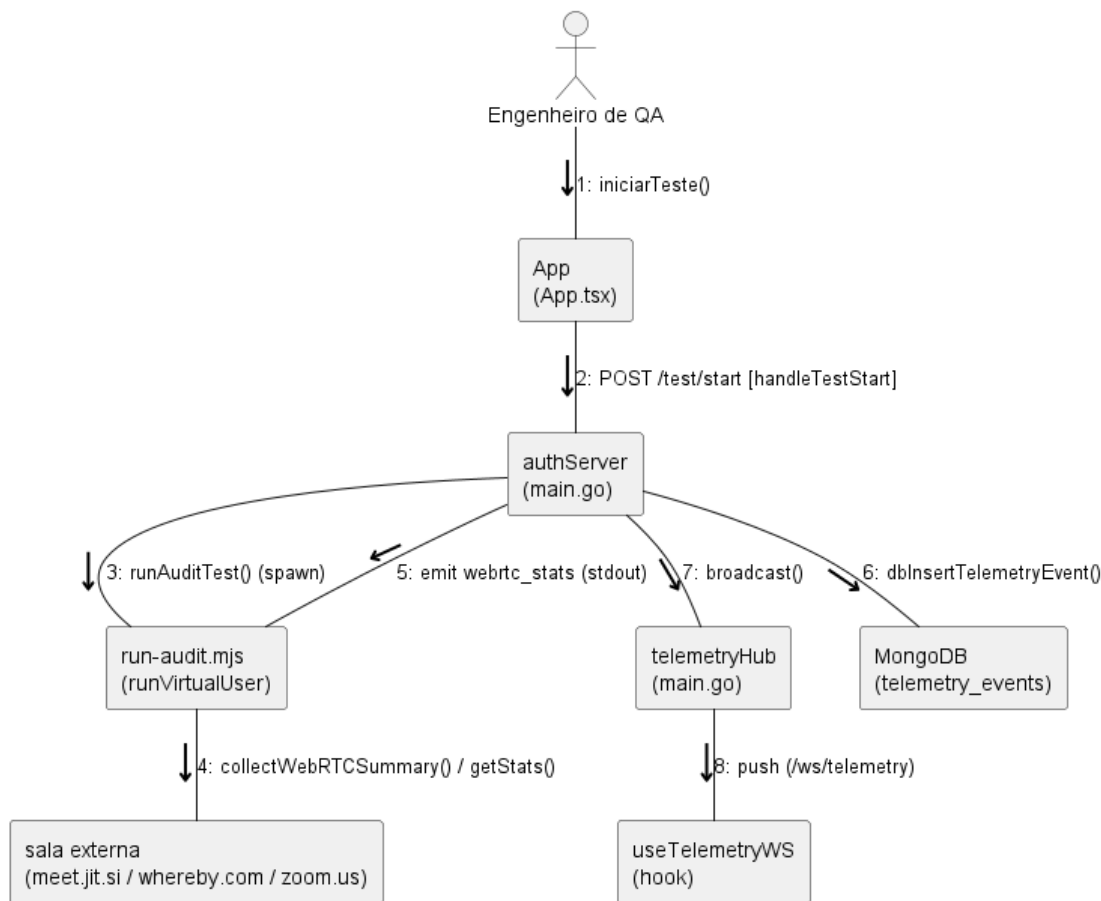


Figura 16: Diagrama de Comunicação de Execução e Telemetria

A Figura 17 apresenta o diagrama de comunicação de Consolidação de Relatório, que detalha o "caminho frio" (*cold path*), focado na consolidação e transformação de dados históricos. O processo é disparado sob demanda pela interface App (em App.tsx, via `handleDownloadReport` — mensagem 1), que aciona o `authServer` (handler `handleExportReports` — mensagem 2: *GET /reports/export*). O `authServer` consulta o MongoDB sobre as coleções *test_history* e *telemetry_events* (mensagem 3: `dbLoadTestHistory()` / `dbLoadSessionEvents()`), agrega os dados em um resumo (mensagem 4: *roll-up*, gerando o `TelemetrySessionSummary`) e aplica a formatação adequada (*JSON* ou *CSV*) antes de transmitir o arquivo por *streaming* (mensagem 5), que é então baixado pelo usuário (mensagem 6). Essa segregação demonstra que a lógica de análise de dados é tratada de forma independente do núcleo de execução, garantindo a integridade e a performance do sistema.

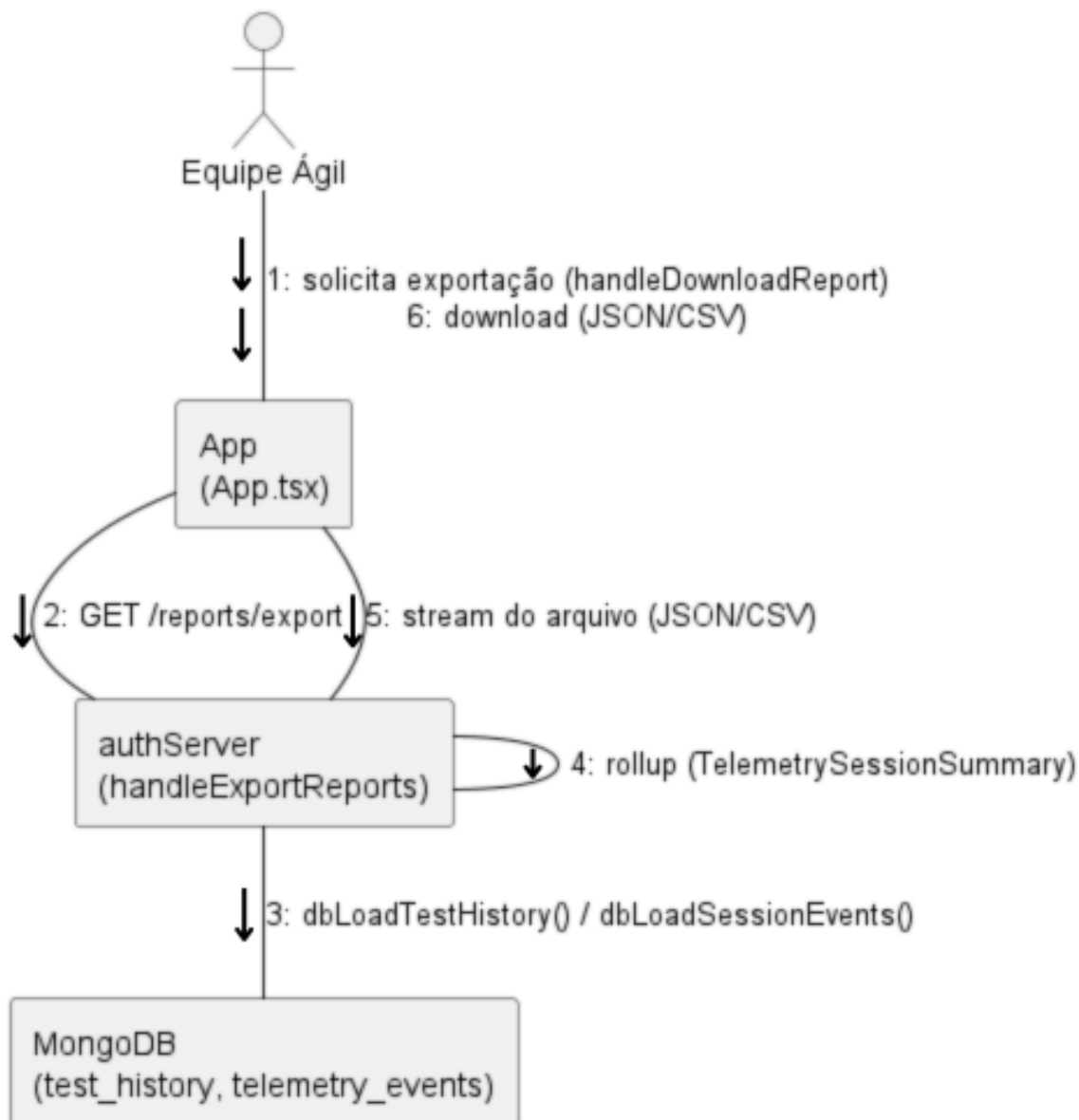


Figura 17: Diagrama de Comunicação de Processamento de Relatórios

A Figura 18 apresenta o diagrama de comunicação do controle ao vivo. As mensagens evidenciam o desacoplamento via arquivo de controle: a interface (App) envia o comando ao authServer, que grava em control.json; o run-audit.mjs lê esse arquivo em intervalos curtos e aplica as alterações no pool e nos navegadores, sem reiniciar o teste. A finalização aciona o cancelamento do processo e a difusão de test_stopped pelo telemetryHub.

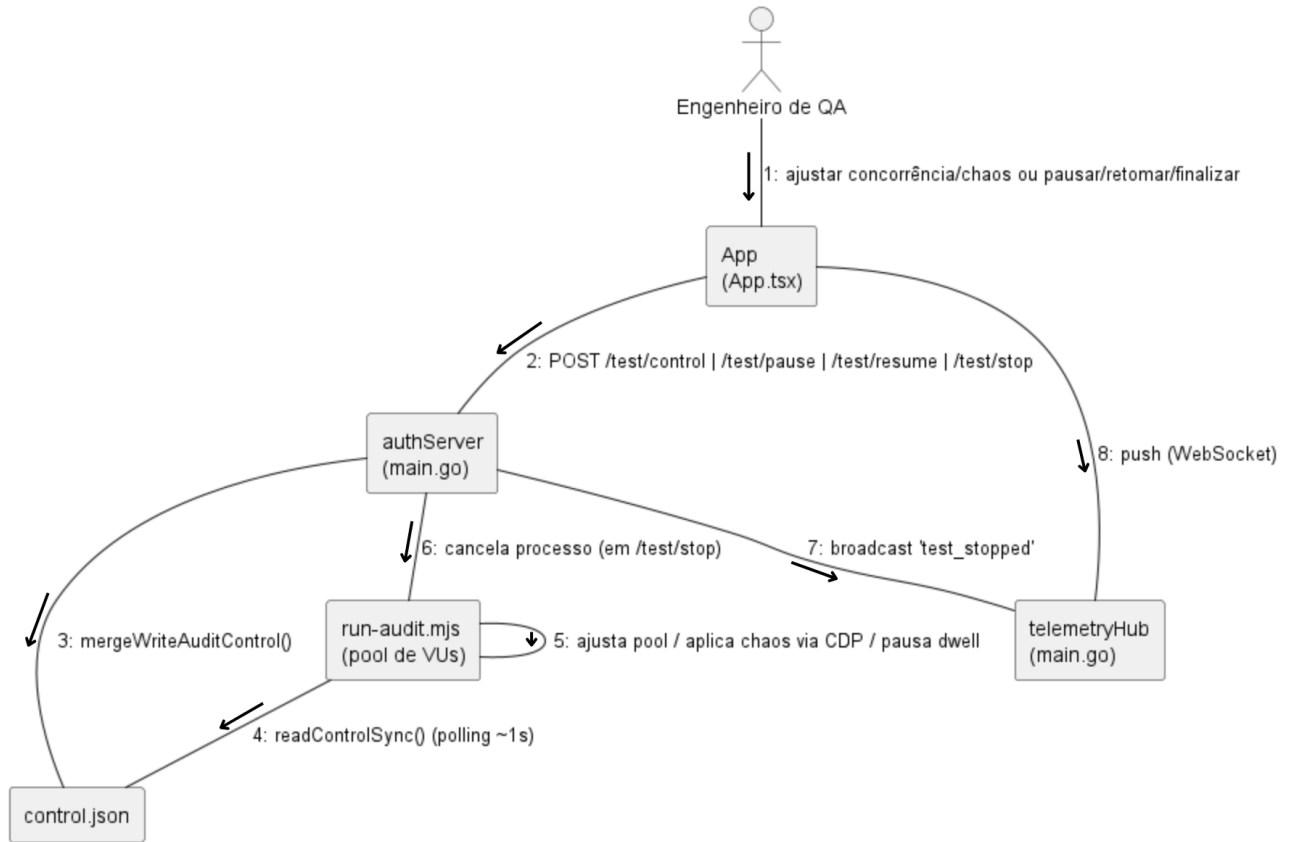


Figura 18: Diagrama de Comunicação de Controle ao Vivo do Teste

A Figura 19 apresenta o diagrama de comunicação da consulta ao histórico, mostrando a interação entre a interface (App), o authServer e o MongoDB para listar as sessões (/tests/history) e reconstruir a auditoria de uma sessão concluída (/reports/session/{id}) a partir das coleções test_history e telemetry_events.

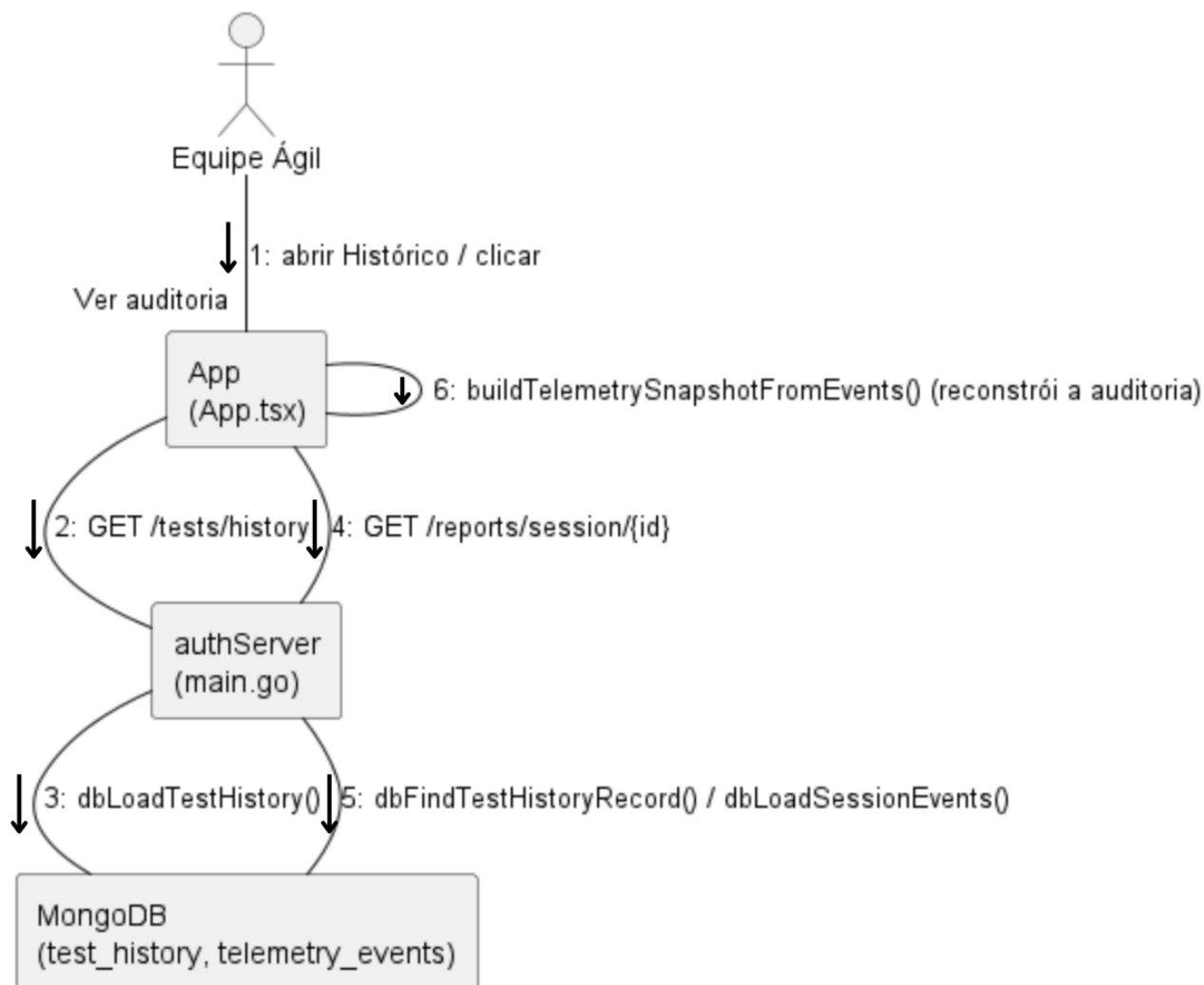


Figura 19: Diagrama de Comunicação de Consulta ao Histórico

3.4 Arquitetura Lógica

A Figura 20 apresenta a arquitetura lógica do Stream Sentry, que organiza o sistema em camadas — Interface Web (React/TypeScript), Servidor *Core* (Go), *Runner* de Execução (Node.js/Puppeteer) e Persistência (MongoDB) —, priorizando o isolamento de processos e a telemetria em tempo real. A

camada de interface gerencia a experiência do usuário por meio de telas como a de Iniciar Teste e a de Auditoria ao vivo, que interagem com o *backend* via *API REST* e *WebSockets*. No Servidor *Core*, o Orquestrador de Teste coordena o ciclo de vida das simulações, acionando o *Runner* (*run-audit.mjs*) para instanciar múltiplos *Workers Puppeteer* que executam as sessões de vídeo e a injeção de instabilidades de rede. A telemetria segue um caminho de dados duplo: o "*Hot Path*" difunde as métricas instantaneamente até a interface por meio do *Hub WebSocket*, enquanto o "*Cold Path*" persiste os eventos no MongoDB para posterior agregação pelo Motor de Relatórios. A integração com os provedores Jitsi, Whereby e Zoom (além de salas *WebRTC* genéricas) ocorre de forma desacoplada através de *drivers* modulares, garantindo escalabilidade e interoperabilidade.

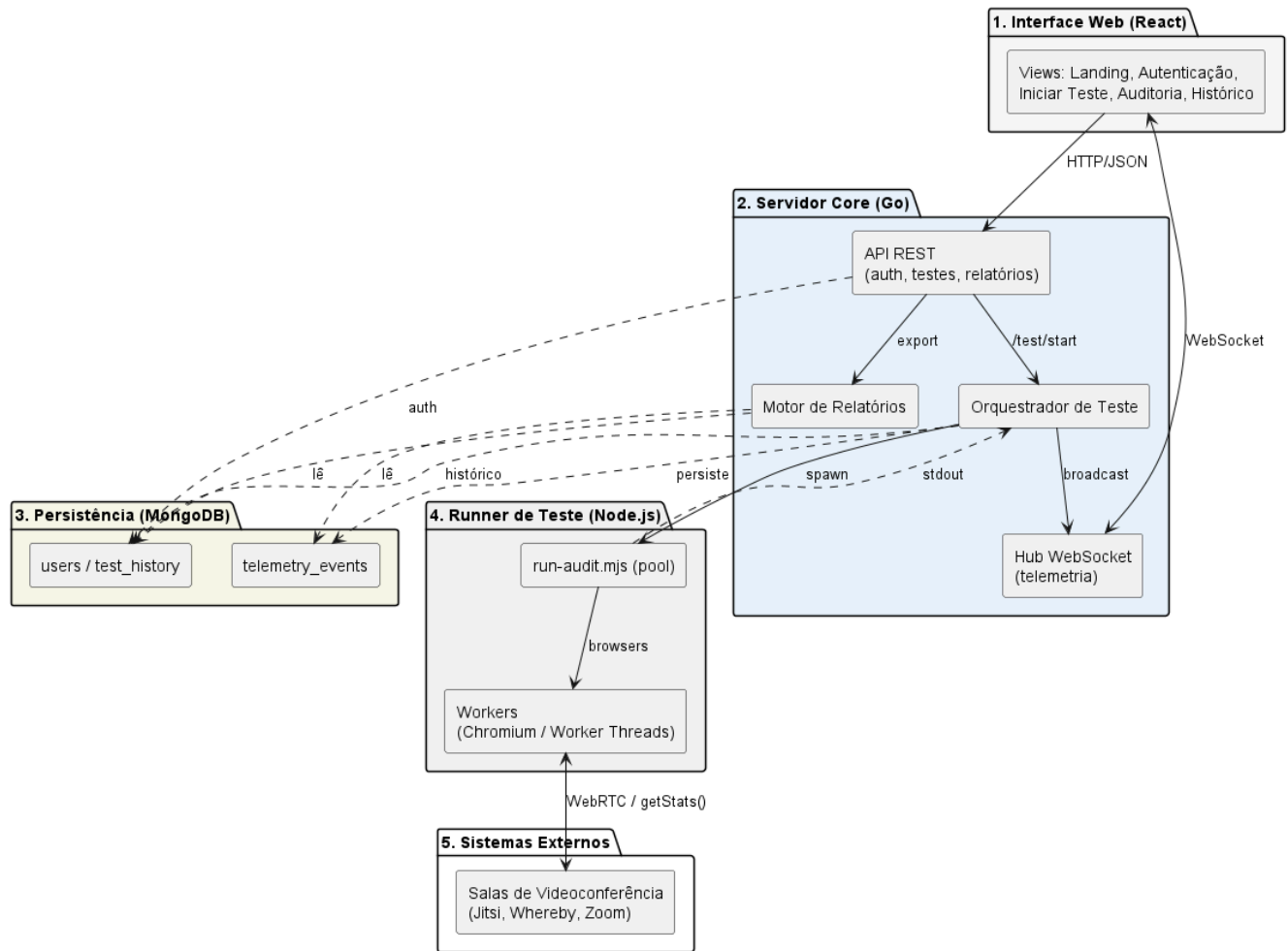


Figura 20: Diagrama de Arquitetura Lógica

3.5 Diagrama de Atividade

O Diagrama de Atividades (Figura 21) detalha o ciclo de vida operacional do Stream Sentry, desde o controle de acesso até a entrega dos resultados analíticos. O fluxo inicia-se obrigatoriamente pela camada de Segurança (UC00), onde a validação de credenciais via *token JWT* libera o acesso ao *Dashboard*. Na fase de Configuração, o sistema valida os parâmetros de estresse e o provedor de vídeo através do *Strategy Pattern* antes da persistência no banco de dados. Durante a Execução, o diagrama destaca o processamento paralelo (*fork*): enquanto o motor de testes orquestra os *Workers Puppeteer* e injeta instabilidades de rede, o sistema gerencia simultaneamente o "*Hot Path*" (telemetria em tempo real via *WebSockets*) e o "*Cold Path*" (persistência de amostras para histórico). Ao atingir o tempo de duração estipulado, o processo de *teardown* é acionado, finalizando as instâncias virtuais e disparando o *ReportGenerator* para a consolidação das métricas de QoE e geração dos relatórios estruturados.

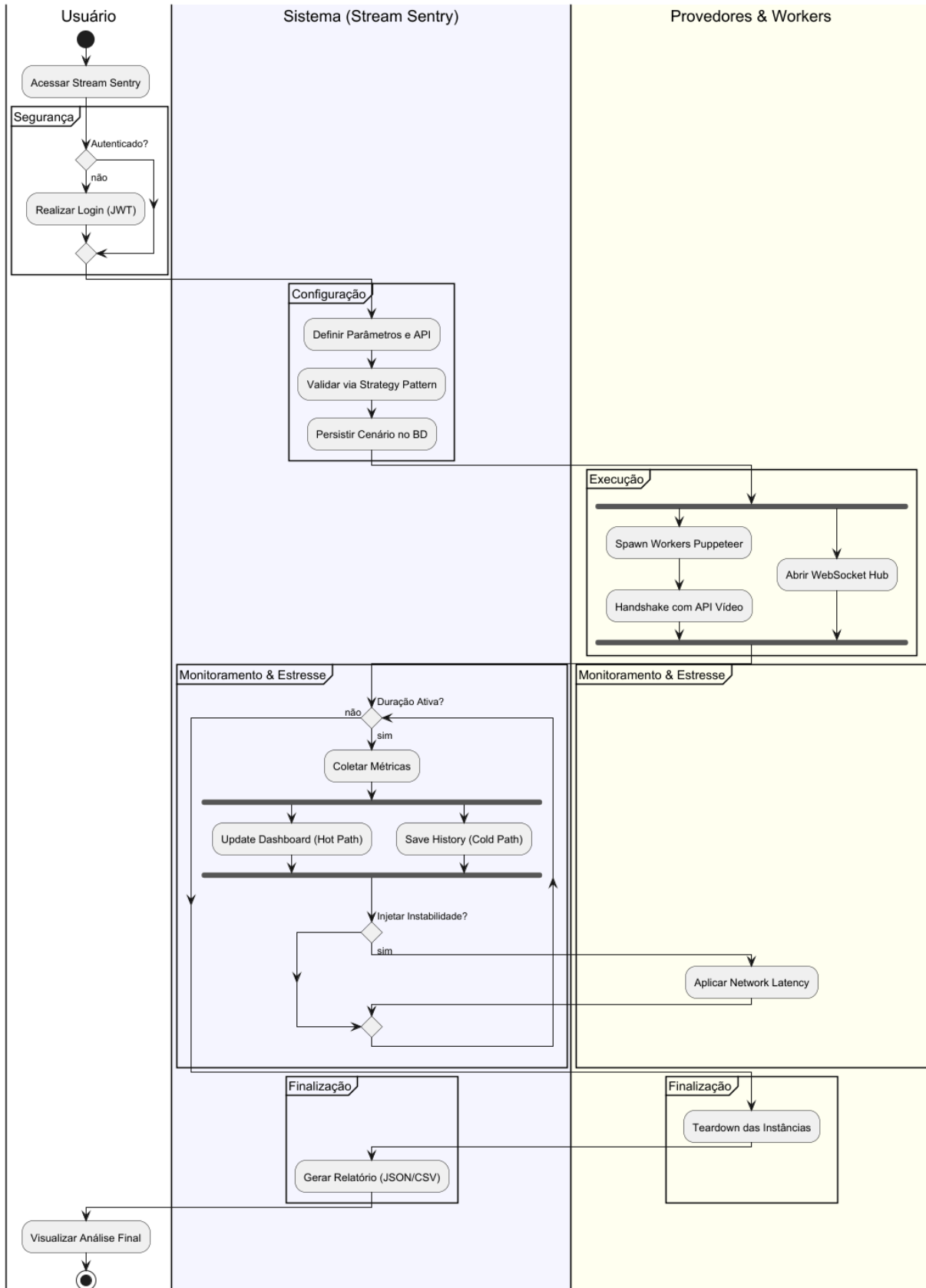


Figura 21: Diagrama de Atividade

3.6 Diagrama de Componentes e Diagrama de Implantação

O Diagrama de Componentes do Stream Sentry (Figura 22) modela os módulos do sistema em quatro blocos. A Interface Web (React/TypeScript) reúne os componentes `App.tsx`, `AuditDashboard.tsx` e o *hook* `useTelemetryWS.ts`. O Servidor *Core* (Go) é composto pelo `authServer` (handlers *REST*), pelo `telemetryHub` (*Hub WebSocket*) e pelo `db.go` (acesso ao MongoDB). O *Runner* (Node.js/Puppeteer) abrange o `run-audit.mjs` e o *ProviderFactory/ConferenceProvider*, que seleciona o *driver* do provedor. Por fim, a Persistência ocorre em MongoDB (coleções *users*, *test_history* e *telemetry_events*). A comunicação entre *frontend* e *backend* se dá pelas interfaces *REST* e *WebSocket*; o Servidor *Core* dispara o *Runner* e persiste a telemetria no MongoDB, enquanto o *Runner* conecta-se aos provedores externos Jitsi, Whereby e Zoom (além de salas *WebRTC* genéricas). Esse desacoplamento por interfaces promove manutenibilidade e escalabilidade.

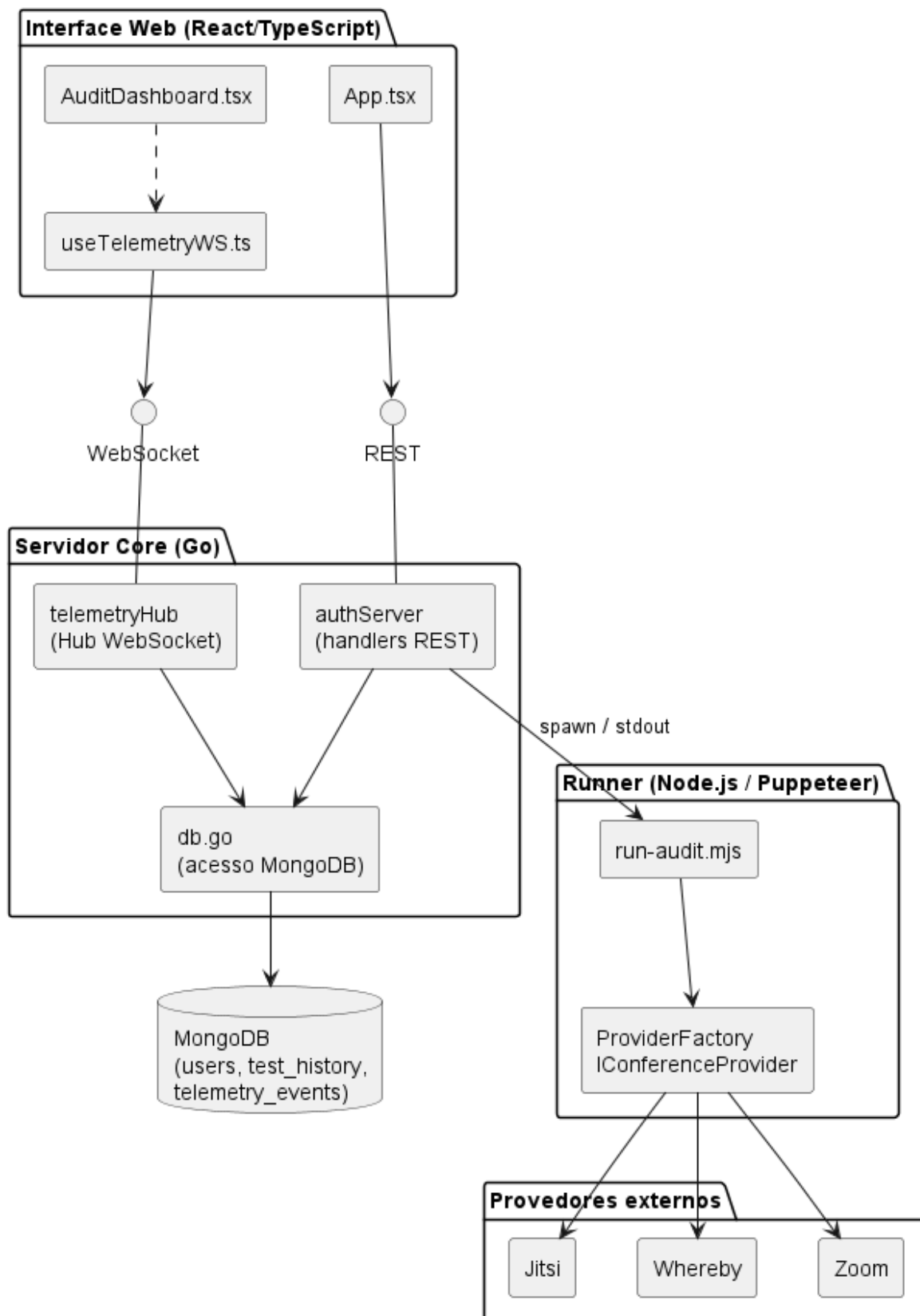


Figura 22: Diagrama de Componentes

O Diagrama de Implantação do Stream Sentry (Figura 23) descreve a infraestrutura de execução. Atualmente o sistema roda em um host único (máquina local ou servidor), que reúne: o *frontend* (React + Vite, *build* estático), o Servidor *Core* em Go (authServer, porta 3001, expondo *REST* e *WebSocket*), o *Runner* em Node.js (run-audit.mjs, que controla instâncias do Chromium via Puppeteer) e o banco MongoDB (porta 27017). O navegador do usuário carrega a *SPA* e comunica-se com o *backend* por *HTTPS* (*REST*) e *WebSocket* (telemetria em tempo real). O authServer persiste os dados no MongoDB e dispara o *Runner*, cujas instâncias do Chromium acessam os provedores externos Jitsi, Whereby e Zoom por *HTTPS/WSS* (*WebRTC*). Para um cenário de produção, os mesmos artefatos podem ser distribuídos — *frontend* estático em um serviço de hospedagem, o *backend* Go com o *runner* em um servidor e um MongoDB gerenciado —, mantendo as mesmas interfaces.

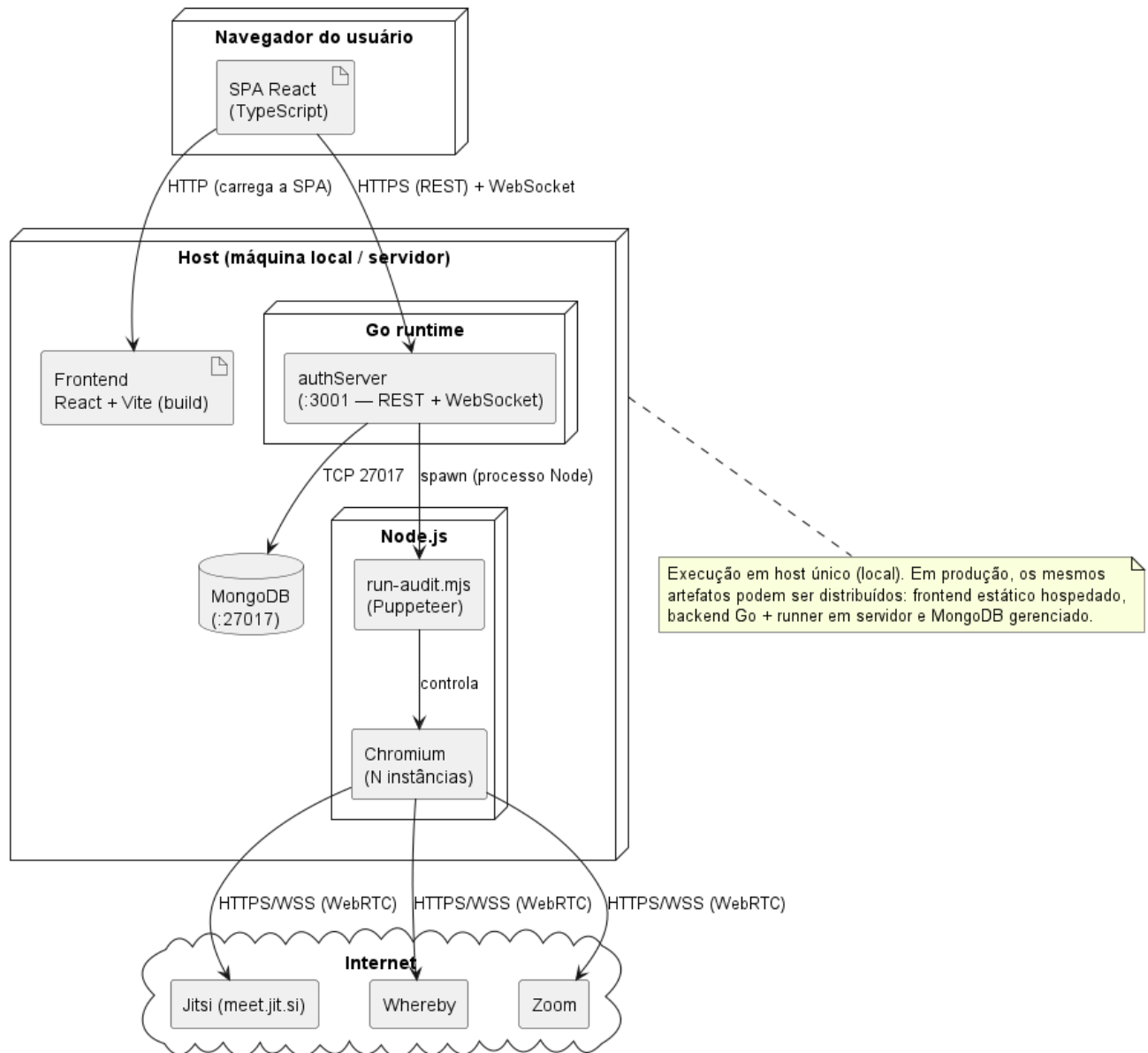


Figura 23: Diagrama de Implantação

4. Projeto de Interface com Usuário

O projeto de interface com usuário do Stream Sentry prioriza clareza técnica, usabilidade ágil e consistência visual, oferecendo uma experiência fluida para todos os atores (desenvolvedores, QA e equipes ágeis). O sistema adota uma interface unificada: todas as telas são comuns a todos os usuários autenticados, sem diferenciação de acesso por perfil. A existência de múltiplas personas, no entanto, justifica-se porque cada funcionalidade é destinada primordialmente a um público específico — embora qualquer usuário possa acessar todas as telas, cada uma atende ao objetivo de um ator distinto: a configuração e execução de testes é voltada ao Desenvolvedor React, que valida a resiliência de suas aplicações; o monitoramento de métricas em tempo real atende ao Engenheiro de QA, responsável por avaliar a qualidade de experiência (QoE); e a consulta ao histórico e a exportação de relatórios servem à Equipe Ágil, que acompanha a estabilidade do produto e integra os dados a seus processos. Assim, as interfaces são comuns no acesso, porém especializadas no propósito. Esta seção está dividida em duas partes: as interfaces projetadas (wireframes concebidos na fase de projeto) e as interfaces implementadas (capturas do sistema em funcionamento), permitindo comparar a concepção inicial com a versão final entregue.

4.1 Interfaces Projetadas (Wireframes) — comuns a todos os atores

A Figura 24 apresenta a tela inicial (*Dashboard*), que oferece uma visão geral das operations e métricas de teste, com um menu superior contendo as opções Configurar Teste, Executar Teste, Relatórios e Documentação, um painel central exibindo gráficos e indicadores de desempenho como latência média, taxa de falhas e cobertura de testes, uma barra lateral com atalhos para configurações avançadas e acesso ao código-fonte, além de um botão de exportação para gerar relatórios em *JSON* ou *CSV*, tudo em um layout minimalista, responsivo e focado em clareza e usabilidade técnica.

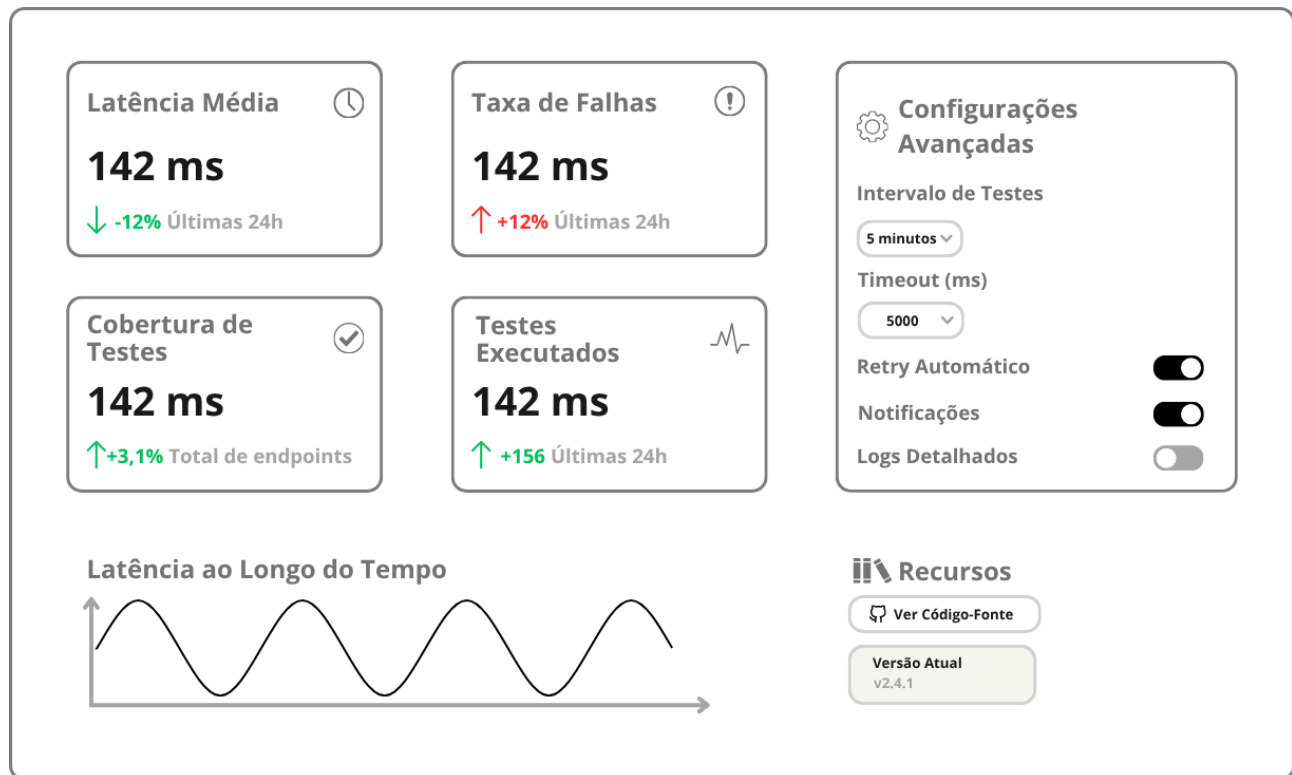


Figura 24: Tela Principal – Dashboard

A Figura 25 apresenta a tela "Configurar Teste", que permite ao usuário definir os parâmetros de um novo teste automatizado, com campos para selecionar a quantidade de usuários virtuais (via controle deslizante), a API de videoconferência (*WebRTC*, Jitsi ou Zoom) e a duração do teste (em minutos), exibindo indicadores de *status* como API conectada e usuários simulados prontos, além de botões "Salvar Configuração" e "Iniciar Teste" para execução direta, tudo em um design simples, com validação rápida de entradas e otimizado para fluxos ágeis.

Status da API
Conexão estabelecida Conectada

Usuários Simulados
Instâncias ativas 5/5

Parâmetros do Teste
Defina a quantidade de usuários virtuais, API e duração do teste de carga

Quantidade de Usuários Virtuais 35

API de Videoconferência
Selecione a API para teste

Duração do Teste
10 minutos

Figura 25: Tela “Configurar Teste”

A Figura 26 apresenta a tela "Executar Teste", responsável por monitorar a execução de testes em tempo real, exibindo um painel de *logs* com informações de desempenho como latência, *bitrate* e falhas, além de um gráfico dinâmico que mostra variações ao longo do tempo, uma barra lateral de progresso indicando o *status* da execução e botões para pausar, finalizar ou gerar relatório, tudo em um visual com tema escuro inspirado em terminais técnicos, transmitindo precisão e controle.

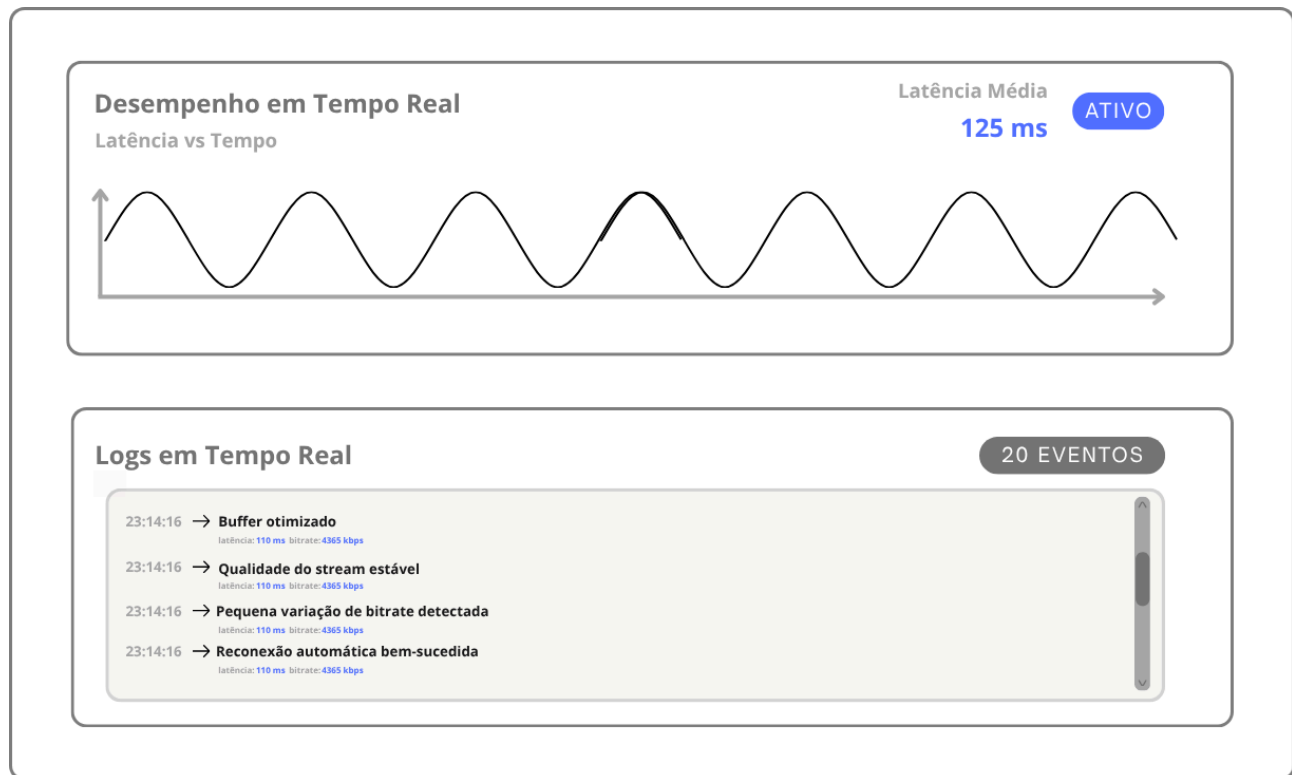


Figura 26: Tela “Executar Teste”

A Figura 27 apresenta a tela "Relatórios", que centraliza o acesso aos resultados dos testes realizados, apresentando uma tabela com histórico de execuções incluindo *API* utilizada, número de usuários e *status* (Sucesso/Falha), exibindo ao selecionar um teste gráficos detalhados com métricas de latência, falhas e qualidade de vídeo/áudio, com opções para filtrar resultados por data, tipo de *API* ou *status* e exportar relatórios em *JSON* ou *CSV*, tudo em um layout de painel analítico com clareza visual e foco na interpretação rápida de dados.

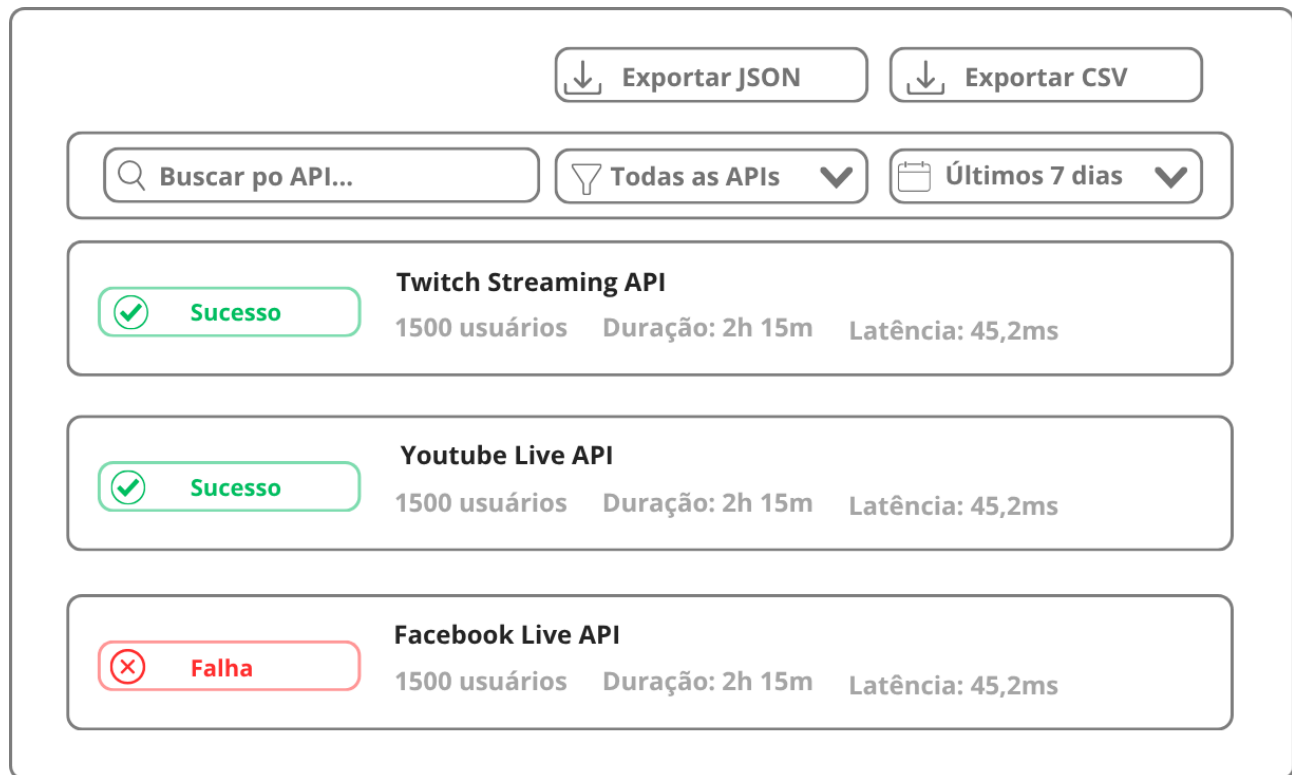


Figura 27: Tela “Relatórios”

A Figura 28 apresenta a tela "Histórico de Testes", que exibe uma linha do tempo vertical com todos os testes realizados, *API* usada, duração e *status* visual (ícones de sucesso/falha), com cada item contendo um *link* direto para o relatório detalhado, em um estilo inspirado em *pipelines* de integração contínua como o GitHub Actions, transmitindo automação e rastreabilidade.

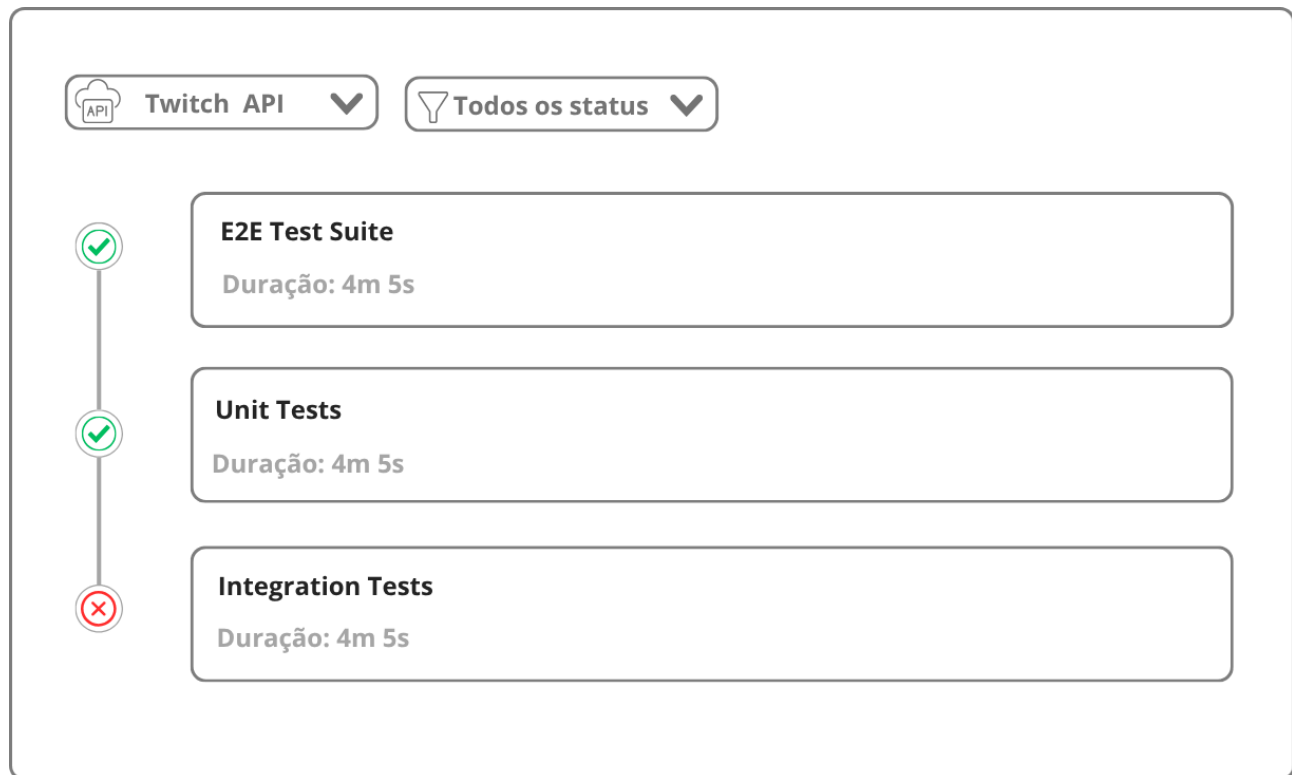


Figura 28: Tela “Histórico de Testes”

A Figura 29 apresenta a tela "Configurações Avançadas", que permite ajustar parâmetros técnicos do sistema, incluindo campos para definir *endpoints* de APIs, *tokens* de autenticação e limites de usuários virtuais simultâneos, com um botão "Salvar" para aplicar as alterações, organizada em blocos separados por categoria para priorizar configuração rápida e segura.

Endpoint da API
Configure o endpoint da API de videoconferência

URL do Endpoint
<https://api.videoconf.example.com/v1>

Token de Autenticação
Configure as credenciais de acesso à API

Token de acesso

Usuários Virtuais
Defina o número máximo de usuários virtuais simultâneos

Limite Máximo	Atual	Em Uso	Disponível
100	100	23	77

Salvar

Figura 29: Tela “Configurações Avançadas”

A Figura 30 apresenta a tela "Login e Registro", com um design minimalista de fundo neutro, logo centralizada e campos de autenticação (email e senha) e confirmação de senha, com botões "Entrar" em estilo uniforme e feedback visual claro, visando facilitar o acesso rápido ao sistema e manter coerência visual com o restante da aplicação.

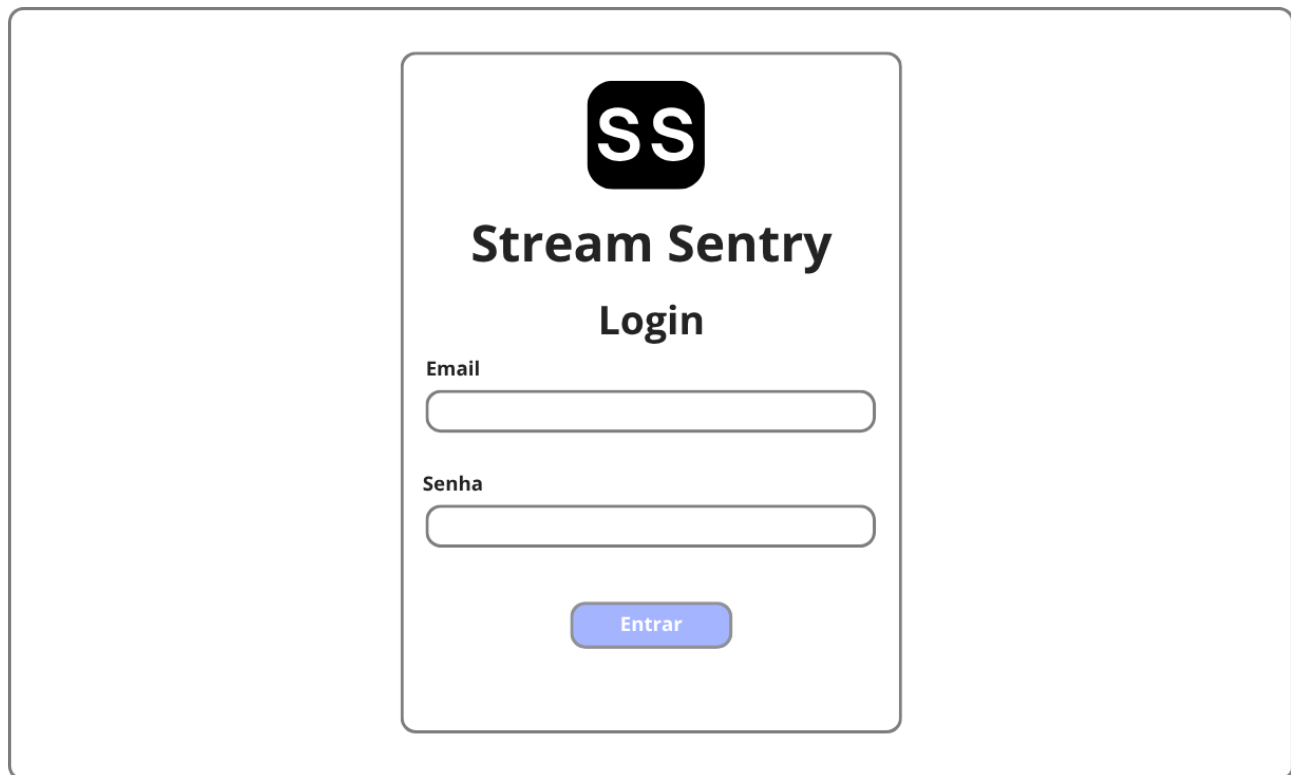


Figura 30: Tela “Login e Registro”

4.2 Interfaces Implementadas

Esta subseção apresenta as telas efetivamente entregues, que evoluíram em relação aos wireframes ao longo do desenvolvimento.

A Figura 31 apresenta a página inicial pública (landing page), exibida antes da autenticação, que traz a identidade do produto (logo e o lema "Observabilidade WebRTC & stress"), a chamada principal "Teste e audite o seu tráfego em tempo real", a lista de recursos (telemetria `getStats()`, até 50 usuários virtuais, perfis de chaos e relatórios exportáveis), a faixa de indicadores (50 usuários simultâneos, 3 plataformas WebRTC, sessões históricas ilimitadas), os cartões de funcionalidades (WebRTC, Rede/chaos, Relatórios e Testes de carga) e os selos das plataformas suportadas (Jitsi Meet, Whereby, Zoom e WebRTC genérico).



Figura 31: Página inicial (Landing Page)

A Figura 32 apresenta a tela de Login/Cadastro: um cartão centralizado, em tema escuro, com a logo, abas para alternar entre "Login" e "Cadastro" e os campos de e-mail e senha (além do nome, no cadastro). O acesso é autenticado por *token JWT*.

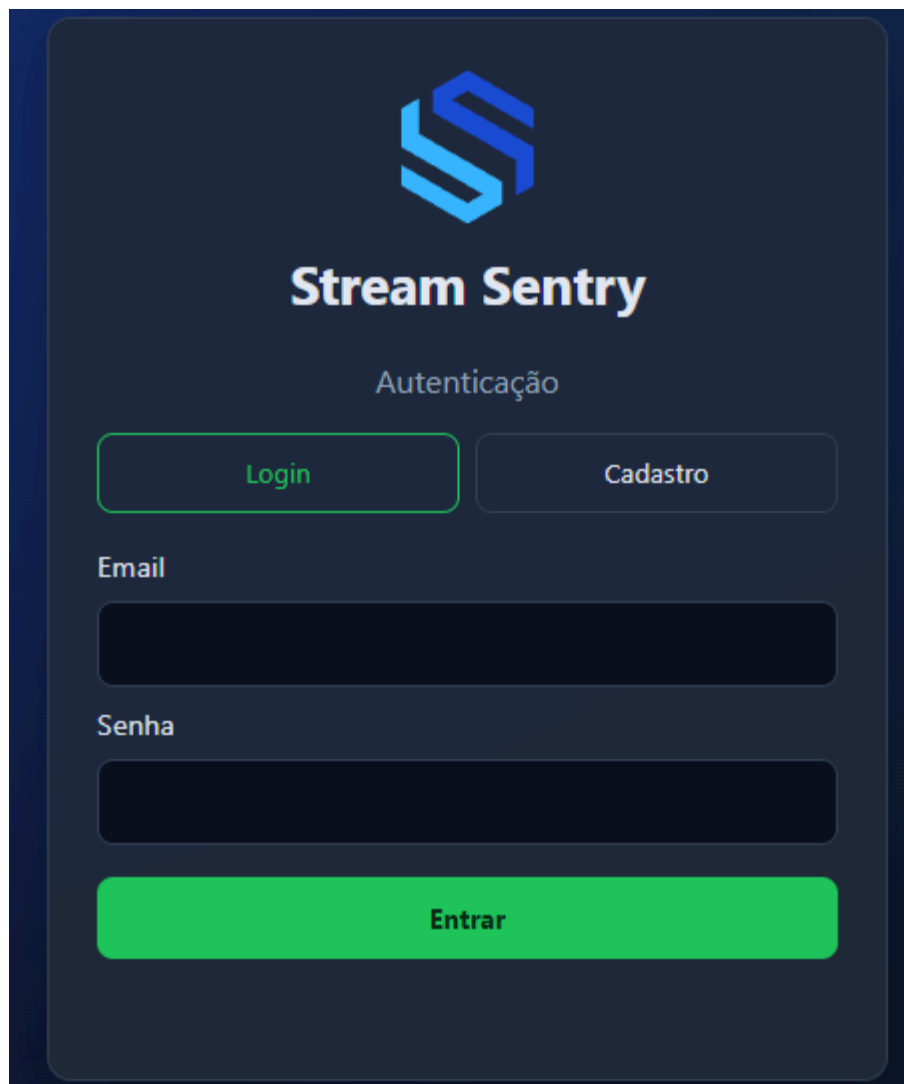
A imagem mostra a interface de autenticação do Stream Sentry. No topo, há o logo do Stream Sentry, que é um 'S' estilizado em azul e verde. Abaixo do logo, o texto 'Stream Sentry' aparece em uma fonte branca e moderna. Segue-se a palavra 'Autenticação' em uma fonte menor. Abaixo disso, há duas abas: 'Login' (destacada com uma borda verde) e 'Cadastro'. Os campos de entrada são rotulados 'Email' e 'Senha' em uma fonte cinza. Cada rótulo é seguido por um campo de entrada retangular escuro. No final da seção, há um botão largo e arredondado de cor verde vibrante com o texto 'Entrar' em branco.

Figura 32: Tela de Login e Cadastro

A Figura 33 apresenta a tela "Iniciar teste", que permite configurar o cenário: a URL alvo (com os botões auxiliares "Criar sala Whereby" e "Gerar sala Jitsi"), o *token* de acesso (opcional para Jitsi e Whereby), a quantidade de usuários virtuais (1 a 50, limitada a 4 no Whereby), a duração da chamada (90 a 1800 segundos), o modo do navegador (*headless/não-headless*) e o perfil de *chaos* de rede. Os botões permitem iniciar o teste, salvar a configuração e executar um *smoke test* do Puppeteer.

Figura 33 : Tela "Iniciar teste"

A Figura 34 apresenta a aba "Auditoria em tempo real", núcleo do sistema. No topo, exibe a pílula de *status* (ex.: "Em execução" → "Finalizado"), a duração do teste e a duração da chamada. O painel de controle ao vivo permite pausar/retomar, ajustar a concorrência (usuários virtuais) e o perfil de *chaos*, com botões de injeção rápida (3G lenta, latência, *offline*, instável). Abaixo, apresenta os indicadores *HTTP* (requisições, respostas, falhas) e os gráficos de Requisições e Atividade (evt/s) com eixo de tempo.



Figura 34: Tela "Auditoria em tempo real"

A Figura 35 apresenta a seção *WebRTC* da Auditoria, alimentada pelas amostras *getStats()*, que agrega os indicadores de *RTT*, *jitter* de vídeo/áudio, usuários com *WebRTC* ativo, *PeerConnections*, pacotes perdidos/recebidos, *frames*, *FPS*, resolução, *bytes* e *bitrate*. Os gráficos por usuário (*RTT*, *jitter* de vídeo, taxa recebida e *FPS*) usam uma cor por usuário virtual (as sete cores do arco-íris para até sete usuários), permitindo comparar o comportamento individual ao longo do teste.



Figura 35: Auditoria – métricas e gráficos WebRTC

A Figura 36 apresenta a aba "Histórico", com a tabela das sessões concluídas (identificador da sessão, início, duração, perfil de *chaos*, *RTT* médio, requisições *HTTP*, amostras *WebRTC* e *status*). Cada linha permite reabrir a auditoria daquela sessão (reconstruída a partir do *log*) e exportar os relatórios em *JSON* ou *CSV*.

**Stream Sentry**
Auditoria e stress WebRTC

Iniciar testeAuditoriaHistórico

Finalizar testeSair

Histórico de testes e relatórios

Cada teste gera um arquivo NDJSON no servidor com **todos os eventos** (rede, WebRTC, latências). O resumo agrega contagens HTTP, estatísticas RTT/jitter e uma série temporal de latência. Use as exportações detalhadas para análise completa ou por sessão.

JSON (resumo)CSV (resumo ampliado)JSON completo (todas)

Sessão	Início	Duração (s)	Chaos	RTT médio	HTTP req	WebRTC amostras	Status	Ações
sess-1781477397174155500	14/06/2026, 19:49:57	105.0	off	—	699	224	Erro	Ver auditoriaJSON+logCSV
sess-1781464797519149800	14/06/2026, 16:19:57	150.3	off	138.5 ms	882	126	OK	Ver auditoriaJSON+logCSV
sess-1781294953480150700	12/06/2026, 17:09:13	109.0	off	162.0 ms	1048	117	OK	Ver auditoriaJSON+logCSV
sess-1781294605320840900	12/06/2026, 17:03:25	109.2	off	145.5 ms	1009	218	OK	Ver auditoriaJSON+logCSV
sess-1781294477632751700	12/06/2026, 17:01:17	107.7	off	—	931	257	OK	Ver auditoriaJSON+logCSV
sess-1781287694843178400	12/06/2026, 15:08:14	109.5	off	153.1 ms	1041	124	OK	Ver auditoriaJSON+logCSV
sess-1781287360401261200	12/06/2026, 15:02:40	322.3	off	146.4 ms	1042	118	Erro	Ver auditoriaJSON+logCSV
sess-1781286879900956900	12/06/2026, 14:54:39	473.3	off	138.4 ms	1041	148	Erro	Ver auditoriaJSON+logCSV
sess-1781285955694753900	12/06/2026, 14:39:15	914.5	off	152.8 ms	875	98	Erro	Ver auditoriaJSON+logCSV
sess-1781285832249773500	12/06/2026, 14:37:12	110.7	off	—	775	220	Erro	Ver auditoriaJSON+logCSV
sess-1781285189011731300	12/06/2026, 14:26:29	560.5	off	145.1 ms	863	133	Erro	Ver auditoriaJSON+logCSV
sess-178128514712	12/06/2026,	27.6	off	—	736	0	Erro	Ver auditoriaJSON+logCSV

Figura 36: Tela "Histórico de testes"

5. Modelos de Projeto

A seção de Modelos de Projeto apresenta os artefatos que detalham a estrutura de dados e os atributos manipulados pelo Stream Sentry, garantindo a consistência entre o projeto e a base de código. Ela é composta pelo modelo de dados — documental, persistido em MongoDB e organizado em coleções que armazenam usuários, o histórico de testes e os eventos de telemetria — e pelo glossário de atributos, que formaliza os campos das entradas (configuração dos testes) e das saídas (relatórios e telemetria), com seus formatos, restrições e descrições. Juntos, esses modelos padronizam o tratamento dos dados e promovem manutenibilidade e escalabilidade, atendendo aos requisitos funcionais e não funcionais do sistema.

5.1 Modelo de Dados

O modelo de dados do Stream Sentry é documental (NoSQL, não relacional), persistido em MongoDB e representado na Figura 37. Diferentemente de um banco relacional — baseado em tabelas, chaves estrangeiras e junções —, o MongoDB armazena documentos (estruturas no formato JSON/BSON, com esquema flexível) organizados em coleções. O modelo é composto pelas coleções principais: `users` (dados de autenticação dos usuários), `test_history` (uma sessão de teste por documento, com o resumo agregado de métricas embutido no campo `summary`), `telemetry_events` (os eventos de telemetria de cada sessão, em formato NDJSON) e `counters` (sequências auxiliares, como o identificador incremental de usuário). O relacionamento principal é por referência: cada documento de `telemetry_events` aponta para a sessão correspondente pelo campo `sessionId` (igual ao `id` de `test_history`), enquanto o resumo da sessão é embutido no próprio documento de `test_history`. Esse modelo documental dispensa junções relacionais, favorecendo a gravação e a leitura rápidas dos eventos de telemetria, e suporta todas as operações do sistema (configurar, executar e relatar).

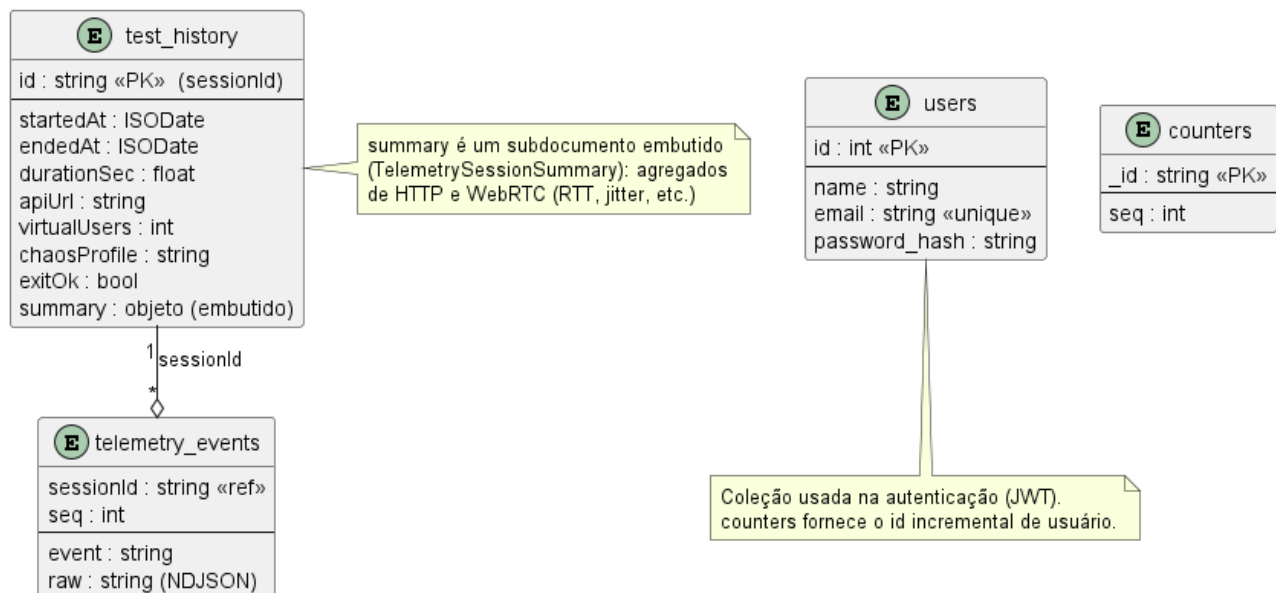


Figura 37: Diagrama de Entidade e Relacionamento

5.2 Glossário

O Glossário de Atributos (Tabela 5) formaliza a definição técnica dos principais campos de dados manipulados pelo Stream Sentry, servindo como referência central para o modelo de dados (coleções do MongoDB), para a validação das entradas e para a interpretação dos relatórios. A tabela abrange desde os parâmetros de configuração do teste (como *virtualUsers*, *apiUrl* e *callDurationSec*) até as métricas de qualidade de experiência (QoE) coletadas em tempo real via *getStats()* (como *rttMs*, *jitterVideo* e *fpsIn*). Para cada atributo são especificados o tipo de dado (*string*, *int*, *float*, *bool*, *ISODate* ou objeto), as restrições de domínio (por exemplo, de 1 a 50 usuários virtuais) e a descrição funcional, padronizando o tratamento das informações que transitam entre o *frontend*, a API em Go e a persistência em MongoDB.

Atributo	Formato	Descrição
apiUrl	<i>string</i>	URL alvo do teste (sala Jitsi/Whereby/Zoom ou página WebRTC genérica).
accessToken	<i>string</i>	Token Bearer enviado nas requisições; opcional para Jitsi e Whereby.
virtualUsers	<i>int</i> (1–50; máx. 4 no Whereby)	Quantidade de usuários virtuais simultâneos.
callDurationSec	<i>int</i> (90–1800)	Duração da chamada por usuário virtual, em segundos.
headful	<i>bool</i>	Modo do navegador (true = janela visível; false = headless).
chaosProfile	<i>string</i> ('off', 'slow_3g', 'fast_3g', 'high_latency', 'offline', 'flaky', 'dns_failure')	Perfil de instabilidade de rede injetado durante o teste.
id (users)	<i>int</i>	Identificador único do usuário.
name	<i>string</i>	Nome do usuário.
email	<i>string</i>	E-mail do usuário (único).
password_hash	<i>string</i>	Hash da senha (bcrypt).

Atributo	Formato	Descrição
id (test_history)	<i>string</i>	Identificador da sessão de teste (ex.: sess-...).
startedAt / endedAt	<i>ISODate</i>	Data e hora de início e término da sessão.
durationSec	<i>float</i>	Duração total da sessão, em segundos.
exitOk	<i>bool</i>	Indica se a sessão encerrou sem erro.
summary	<i>objeto</i>	Resumo agregado da sessão (métricas HTTP e WebRTC).
sessionId	<i>string</i>	Sessão à qual um evento de telemetria pertence (coleção telemetry_events).
seq	<i>int</i>	Número de ordem do evento de telemetria na sessão.
event	<i>string</i>	Tipo do evento (ex.: webrtc_stats, request, test_start).
raw	<i>string</i>	Linha NDJSON original do evento, preservada integralmente.
rttMs	<i>float (ms)</i>	Tempo de ida e volta (round-trip time) dos pacotes WebRTC.
jitterVideo / jitterAudio	<i>float (ms)</i>	Variação no tempo de chegada dos pacotes RTP de vídeo/áudio.
packetsReceived / packetsLost	<i>int</i>	Pacotes RTP recebidos e perdidos.
framesDecoded	<i>int</i>	Frames de vídeo decodificados.
fpsIn / fpsOut	<i>float</i>	Quadros por segundo do vídeo recebido/enviado.
resolution	<i>string</i>	Resolução do vídeo recebido (ex.: 320×180).
inboundBytes / outboundBytes	<i>int</i>	Volume de dados RTP recebidos/enviados, em bytes.
availableOutgoingBitrate	<i>float (kbps)</i>	Estimativa de banda disponível para envio.

Atributo	Formato	Descrição
peerConnections	<i>int</i>	Número de RTCPeerConnections ativas.

Tabela 5: Glossário

6. Plano de Testes

6. Plano de Testes Esta seção detalha a estratégia de garantia de qualidade do Stream Sentry, dividida em testes de aceitação, focados na validação das necessidades de negócio, e testes de integração, focados na comunicação entre os componentes do sistema e os serviços externos.

6.1 Plano de Teste de Aceitação

O Plano de Teste de Aceitação valida se as funcionalidades entregues correspondem às necessidades de negócio do sistema. A primeira necessidade refere-se à Configuração Flexível e Automação de Testes, cujo objetivo é garantir a parametrização correta para diferentes provedores, respeitando restrições de entrada e segurança. Conforme a Tabela 6, o cenário TA-01 verifica o fluxo positivo de uma configuração válida no Jitsi; o TA-02 aplica teste de fronteira (*Boundary Value*) sobre o limite de 50 usuários virtuais (e de 4 no Whereby); e o TA-03 verifica o tratamento de *token* de acesso inválido em um alvo autenticado.

ID	Caso de Teste	Pré-condição	Entrada de Dados	Saída Esperada / Critério de Aceite
TA-01	Configuração de teste Jitsi com sucesso	Usuário autenticado	URL: sala Jitsi (botão "Gerar sala Jitsi"); Usuários: 5; Duração da chamada: 120 s; Ação: "Iniciar teste"	O sistema valida os campos, vai para a aba Auditoria com status "Em execução"; a janela do VU 1 abre para login manual no Google e os demais entram como convidados; a telemetria começa a chegar.
TA-02	Validação do limite de usuários (Boundary Value)	Usuário na tela "Iniciar teste"	URL: WebRTC genérico; Usuários: 51	O início é bloqueado com a mensagem "O número de usuários deve estar entre 1 e 50". (No Whereby, o campo é limitado a 4 e redefinido para 1 ao selecionar a sala.)

TA-03	Token de acesso inválido	Alvo (API genérica) que exige autenticação Bearer	Token inválido; Ação: "Iniciar teste"	As requisições ao alvo retornam 401 Unauthorized, contabilizadas no indicador "Falhas de rede" e registradas no feed da Auditoria.
-------	--------------------------	---	---------------------------------------	--

Tabela 6: Configuração e Automação

A segunda necessidade foca no Monitoramento de Qualidade e Coleta de Métricas, assegurando a precisão da telemetria em tempo real e a reação a instabilidades. Conforme a Tabela 7, o TA-04 injeta latência via *chaos* e confirma a reação nos gráficos; o TA-05 verifica a contabilização de falhas diante de perdas/quedas de rede; e o TA-06 valida o encerramento automático ao fim da duração configurada.

ID	Caso de Teste	Pré-condição	Entrada de Dados	Saída Esperada / Critério de Aceite
TA-04	Reação a latência elevada (<i>chaos</i>)	Teste em execução	Aplicar o perfil de <i>chaos</i> "Alta latência" pela Auditoria	O gráfico de RTT sobe e passa a exibir a etiqueta "Alta latência (RTT)"; os indicadores WebRTC refletem o aumento.
TA-05	Contabilização de falhas de rede	Teste em execução com vários usuários	Aplicar o perfil "Offline" ou "Instável" (<i>chaos</i>)	O indicador "Falhas de rede" é incrementado e os eventos aparecem no feed; o teste prossegue para os demais usuários sem ser abortado.
TA-06	Encerramento automático e consistência	Teste com duração da chamada de 90 s	Aguardar o término da duração	Ao atingir a duração, o pool entra em drenagem, encerra os usuários virtuais, o status muda para "Finalizado" e os dados ficam disponíveis na Auditoria e no Histórico.

Tabela 7: Monitoramento de Qualidade

A terceira necessidade cobre a Análise de Dados e Relatórios, validando a persistência e a integridade dos dados históricos. Conforme a Tabela 8, o TA-07 verifica a integridade da exportação

JSON; o TA-08 confirma a persistência da sessão no Histórico; e o TA-09 valida a reabertura da auditoria histórica a partir do *log*.

ID	Caso de Teste	Pré-condição	Entrada de Dados	Saída Esperada / Critério de Aceite
TA-07	Integridade da exportação <i>JSON</i>	Sessão concluída	Aba Histórico; Ação: " <i>JSON</i> (resumo)" ou " <i>JSON</i> completo"	Download de um <i>JSON</i> válido contendo os metadados da sessão (id, startedAt, durationSec, apiUri, chaosProfile) e o <i>summary</i> agregado (<i>HTTP</i> e <i>WebRTC</i>); na opção completa, também os eventos de telemetria.
TA-08	Persistência no Histórico	Sessão recém-concluída (TA-06)	Ação: navegar até a aba Histórico	A sessão aparece na lista exibindo corretamente início, duração, perfil de chaos, RTT médio, requisições <i>HTTP</i> , amostras <i>WebRTC</i> e status.
TA-09	Reabertura de auditoria histórica	Existe ao menos uma sessão concluída	Ação: clicar em "Ver auditoria" na linha da sessão	A tela de Auditoria é reconstruída a partir do <i>log</i> (telemetry_events), reexibindo os KPIs e os gráficos daquela sessão.

Tabela 8: Análise e Relatórios

6.2 Plano de Teste de Integração

O teste de integração foca na comunicação entre os módulos do Stream Sentry (*backend* em Go, *runner* em Node.js e MongoDB), utilizando a estratégia *Incremental Bottom-Up* para validar persistência e lógica antes da interface. Para garantir ambientes limpos e isolados, utiliza-se Docker para o banco de dados e *mocks* para provedores externos (Jitsi, Whereby e Zoom) no *pipeline* de Integração Contínua (CI). As interfaces e cenários validados — comunicação *REST*, persistência e canais *WebSocket/WebRTC* — estão detalhados na Tabela 9.

ID	Interface Envolvida	Descrição do Cenário	Validação Técnica
TI-01	Backend ↔ MongoDB	Persistência da sessão e da telemetria	Iniciar um teste via POST /test/start; verificar que os eventos são gravados em telemetry_events durante a execução e que, ao final, a sessão é registrada em test_history (consulta direta db.test_history.findOne({ id })).
TI-02	Runner ↔ Provedor (Jitsi/Whereby)	Ingresso na sala de vídeo	O runVirtualUser (run-audit.mjs) entra na sala e o collectWebRTCSummary() passa a retornar peerConnections > 0, com emissão de eventos webrtc_stats contendo atividade.
TI-03	Frontend ↔ Backend (WebSocket)	Atualização de métricas ao vivo	O telemetryHub difunde o evento webrtc_stats em /ws/telemetry; verificar que o hook useTelemetryWS atualiza o estado e que o gráfico renderiza o novo ponto.
TI-04	Frontend ↔ Backend ↔ MongoDB	Fluxo completo de relatório	Requisição GET /reports/export?sessionId=...&format=json; o backend lê test_history e telemetry_events no MongoDB, serializa o conteúdo e o frontend realiza o download sem erros.

Tabela 9: Integração e Estratégia

7. Cronograma e Processo de Implementação

Esta seção descreve a abordagem de engenharia de software adotada para a construção do *Stream Sentry*, detalhando a metodologia, as ferramentas de controle, o *pipeline* de integração contínua (CI/CD) e o planejamento temporal para garantir a entrega do projeto dentro do prazo estipulado.

7.1 Cronograma

O cronograma do TCC II foi executado em um modelo Ágil, estruturado em dez Sprints de duas semanas cada, cobrindo o desenvolvimento completo do software e a documentação técnica até o final de junho de 2026, conforme detalhado na Tabela 10. Este planejamento priorizou o objetivo central do TCC: as Sprints iniciais (S1 a S5) focaram no Core, métricas e integrações com as APIs de vídeo; as Sprints seguintes (S6 a S8) dedicaram-se à Interface Web e aos Testes Integrados (E2E); e as Sprints finais (S9 e S10) foram reservadas à coleta de métricas definitivas e à redação dos capítulos de resultados da monografia, garantindo que o produto técnico e os dados para o TCC fossem consolidados no prazo.

Mês	Período (Quinzenas)	Sprint	Foco Principal	Atividades Detalhadas
Fev/26	1ª Quinzenal (1-15)	S1	Setup e Core Base	Configuração do ambiente (Node.js/PostgreSQL). Implementação dos modelos de dados e do <i>TestController</i> .
	2ª Quinzenal (16-29)	S2	Motor de Testes	Implementação do <i>TestEngine</i> e <i>MetricCollector</i> base. Primeira execução funcional do teste com <i>WebRTC Native</i> .
Mar/26	1ª Quinzenal (1-15)	S3	Coleta de Métricas	Finalização do <i>MetricCollector</i> . Implementação das rotinas de coleta de latência e taxa de falhas e persistência no <i>BD</i> .
	2ª Quinzenal (16-31)	S4	Integração de APIs	Implementação da conexão com Zoom e Jitsi SDK . Validação da coleta de métricas nas APIs integradas.
Abr/26	1ª Quinzenal (1-15)	S5	Relatórios RAW e Docs	Implementação do <i>ReportGenerator</i> . Geração do Relatório em formato RAW (JSON/CSV) . Finalização do Capítulo 3 (Metodologia).

	2ª Quinzenal (16-30)	S6	Interface Essencial	Desenvolvimento das telas Configurar Teste e Executar Teste (monitoramento em tempo real) no React.
Mai/26	1ª Quinzenal (1-15)	S7	Visualização de Dados	Desenvolvimento das telas Relatórios e Dashboard no React, incluindo visualização de gráficos e exportação de dados.
	2ª Quinzenal (16-31)	S8	Testes Integrados (E2E)	Execução da primeira rodada de Testes Integrados (E2E) completos em todos os fluxos e APIs. Correção de <i>bugs</i> críticos.
Jun/26	1ª Quinzenal (1-15)	S9	Qualidade e Docs Técnica	Segunda rodada de Testes Integrados . Elaboração da Documentação Técnica (Arquitetura, APIs) e do Capítulo 4 (Resultados/Análise V1).
	2ª Quinzenal (16-30)	S10	Fechamento e Resultados Finais	Fechamento da Versão 1.2. Coleta de Métricas Definitivas para o TCC. Finalização do Capítulo 5 (Conclusão e Trabalhos Futuros).

Tabela 10: Cronograma

7.2 Metodologia de Desenvolvimento

A gestão e o rastreamento das tarefas são realizados no GitHub Projects, em formato *Kanban*, com o progresso das atividades distribuído entre as colunas "*Backlog*", "*Ready*", "*In progress*", "*In review*" e "*Done*". Esse quadro centraliza a visualização do andamento do trabalho e garante a rastreabilidade das atividades ao longo do desenvolvimento. A Figura 38 apresenta o quadro de tarefas do projeto, com todas as atividades já concluídas (coluna "*Done*").

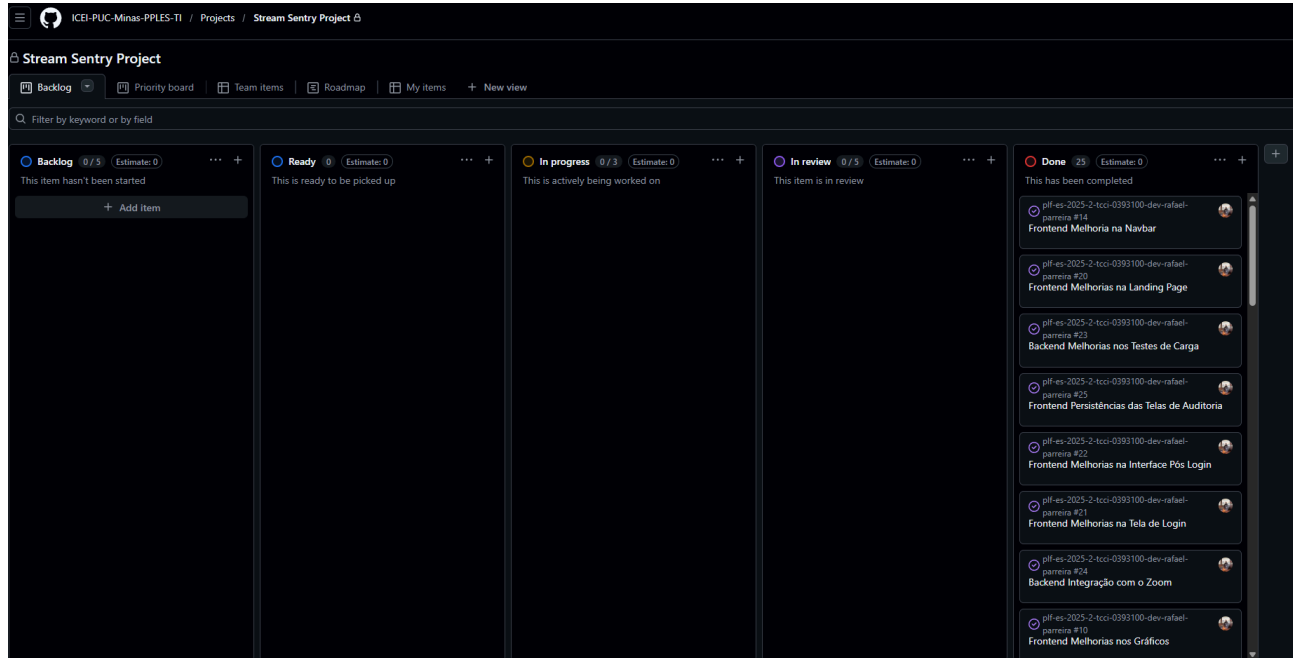


Figura 38: Quadro de tarefas no GitHub Projects (Kanban)

O trabalho foi planejado em quatro *milestones*, correspondentes às *sprints* 1 a 4, que organizam as entregas incrementais do projeto e marcam os principais pontos de progresso do cronograma. A Figura 39 apresenta a divisão dessas *milestones* e seu respectivo progresso, todas concluídas ao final do desenvolvimento.



Figura 39: Divisão de milestones (sprints 1 a 4) do projeto

7.3 Ferramentas e Pipeline de CI/CD

Para assegurar a confiabilidade do software, foi estabelecido um *pipeline* de Integração Contínua (CI) com a plataforma GitHub Actions, executado automaticamente a cada *push* ou *pull request* na *branch* principal. O *pipeline* é dividido em dois *jobs*. No *job* de *backend*, o código Go passa por análise estática (*go vet*) e pela bateria de testes unitários (*go test*), que cobre a validação de configuração do teste, a extração de identidade a partir do *token JWT* e a agregação de telemetria (*rollup* de métricas). No *job* de *frontend*, o código React/TypeScript passa pela verificação estática de tipos (*tsc --noEmit*), pelos testes unitários com Vitest (detecção de provedor e validação dos parâmetros do teste) e pela compilação de produção (*Vite build*). Assim, erros de tipo, falhas de lógica e quebras de *build* são detectados antes da integração ao ramo principal. A etapa de Entrega Contínua (CD) — a implantação automatizada do *frontend* e do *backend* em ambiente de hospedagem — está prevista como trabalho futuro, quando o sistema for publicado; atualmente a execução ocorre em ambiente local.

8. Post Mortem

Esta seção apresenta uma análise retrospectiva do desenvolvimento do Stream Sentry ao longo do TCC II, registrando os acertos, as dificuldades e os aprendizados consolidados durante a implementação. O foco recai sobre os principais desafios técnicos enfrentados na integração com os provedores de videoconferência (Zoom, Whereby e Jitsi) por meio do padrão *Strategy (IConferenceProvider / ProviderFactory)*, bem como sobre as decisões de arquitetura tomadas em resposta a limitações externas, fora do controle do projeto. Cabe registrar que todos os provedores foram avaliados em suas versões/planos gratuitos, sem contratação de licenças pagas; parte das limitações relatadas a seguir — em especial restrições de capacidade — decorre diretamente dessa condição, e poderia ser atenuada ou eliminada em planos comerciais. A retrospectiva está organizada em quatro frentes: experiências positivas, experiências negativas, lições aprendidas e o repositório do trabalho.

8.1 Experiências Positivas

A principal experiência positiva foi a validação prática do desacoplamento arquitetural proposto desde a concepção do sistema. A adoção do padrão *Strategy/Factory* permitiu que a troca do provedor de referência durante o desenvolvimento — do Zoom para o Whereby e o Jitsi — fosse absorvida sem necessidade de alterar o núcleo do *TestEngine*. Essa flexibilidade foi decisiva para a continuidade do projeto diante dos bloqueios encontrados.

A integração com o Whereby superou as expectativas no que tange à automação. Por ser uma plataforma baseada em *WebRTC* nativo e com interface web mais permissiva, foi possível automatizar integralmente o fluxo de criação de sala, ingresso dos usuários virtuais e coleta de telemetria, sem qualquer intervenção manual, *login* ou contorno de mecanismos anti-bot. Os *Workers Puppeteer* ingressam na sala como convidados, habilitam câmera e microfone simulados e iniciam a coleta de estatísticas *WebRTC* de forma consistente para todos os participantes, cumprindo plenamente o objetivo central do TCC (US02).

O Jitsi Meet, por sua vez, foi o provedor que mais se aproximou do cenário ideal descrito no Documento de Visão: suportou a simulação de até 50 usuários virtuais simultâneos, conforme prometido nos requisitos não funcionais e validado no caso de teste TA-01, com coleta completa de métricas *WebRTC*. Por fim, a estratégia de telemetria — injeção de um *hook* em *RTCPeerConnection* e amostragem periódica de *getStats()* no estilo *webrtc-internals* — mostrou-se robusta e fiel aos dados de qualidade de experiência (QoE) expostos pelo navegador, sustentando a auditoria em tempo real via *WebSocket*.

8.2 Experiências Negativas

A experiência mais frustrante foi a inviabilização do Zoom como provedor de referência para os testes automatizados. Durante a implementação dos *Workers Puppeteer*, foram identificados dois problemas críticos:

- Bloqueio de acesso a *bots*: o *Zoom Web Client* possui mecanismos de detecção de automação (*fingerprinting* de navegador, *CAPTCHA* e verificações de comportamento) que identificam sessões controladas pelo *Puppeteer* e bloqueiam o ingresso na reunião. Mesmo com técnicas de mitigação — uso de *ghost-cursor* para simular movimentação humana do mouse, perfis de navegador persistentes e *user agents* customizados —, o acesso automatizado às salas via *zoom.us/j/...* mostrou-se inconsistente e frequentemente bloqueado.
- Coleta parcial de métricas mesmo no acesso manual: ao contornar o bloqueio com ingresso manual, constatou-se que o *Zoom Web Client* não expõe a totalidade das estatísticas de *WebRTC* esperadas — aproximadamente metade dos indicadores de QoE definidos no Glossário, como *jitterVideo* e *downlinkKbps* para os fluxos de entrada de outros participantes, retornavam valores nulos ou zerados, comprometendo a integridade dos relatórios analíticos e a comparabilidade entre provedores.

No Whereby, a limitação foi de natureza comercial, e não técnica: o plano gratuito restringe as salas a um máximo de 4 usuários simultâneos por chamada, impondo um teto ao volume de usuários virtuais simulável em uma única sessão, em contraste com a meta de 50 usuários do sistema (TA-02). Sua superação dependeria da contratação de um plano pago (Whereby Embedded/Business), fora do escopo orçamentário do TCC.

No Jitsi, a dificuldade esteve na automação do ingresso quando a sala exige um anfitrião autenticado. O *login* automático (*JITSI_AUTH_**, formulário *in-page*) mostrou-se frágil diante de fluxos de autenticação em dois fatores (2FA) e telas de confirmação dinâmicas do Google, exigindo uma intervenção manual pontual na primeira execução.

8.3 Lições Aprendidas

A primeira lição foi a confirmação de que o desacoplamento arquitetural não é um luxo, mas uma necessidade prática. Sem o padrão *Strategy/Factory*, a substituição do provedor de referência teria exigido reescrever partes significativas do motor de execução; com ele, a mudança ficou isolada nos *drivers*, preservando o núcleo do sistema.

A segunda lição foi a importância de validar precocemente a viabilidade de automação de plataformas externas. Plataformas comerciais como o Zoom investem ativamente em mecanismos anti-automação, e essa restrição — somada à exposição incompleta de estatísticas *WebRTC* — só se revelou plenamente durante a implementação. Em projetos futuros, uma prova de conceito de

ingresso automatizado e coleta de métricas deveria preceder a escolha definitiva do provedor de referência.

A terceira lição foi a necessidade de distinguir limitações técnicas de limitações comerciais. Enquanto o bloqueio do Zoom é uma barreira técnica de difícil contorno, o limite de 4 usuários do Whereby é uma restrição de licenciamento contornável com investimento — distinção relevante para o planejamento e para a comunicação honesta das capacidades do sistema. Como contorno para o Jitsi, adotou-se uma solução pragmática: apenas o usuário virtual 1 (VU1) abre uma janela de navegador visível para *login* manual do anfitrião na primeira execução, enquanto um perfil *Chrome* persistente (*JITSI_CHROME_PROFILE_DIR*) reaproveita a sessão nas execuções seguintes e os demais usuários ingressam como convidados de forma totalmente automatizada — solução que indica, ainda, uma oportunidade de melhoria futura (uso de contas de serviço dedicadas ou salas que dispensem anfitrião autenticado).

Em síntese, consolidou-se o seguinte panorama dos provedores:

- Zoom: integrado e utilizável manualmente, porém inviável para automação (bloqueio anti-bot) e com coleta parcial de métricas mesmo no uso manual. Status: integrado, porém não recomendado.
- Whereby: automação completa, sem intervenção manual e com métricas *WebRTC* completas; limitado a 4 usuários no plano gratuito. Status: recomendado para testes de pequena escala.
- Jitsi: automação quase completa (anfitrião requer login manual na primeira execução), métricas *WebRTC* completas e suporte a até 50 usuários. Status: recomendado para testes de escala.

8.4 Repositório do Trabalho

O código-fonte produzido, a documentação técnica e os artefatos de modelagem estão disponíveis publicamente no repositório GitHub do projeto, sob licença MIT:

<https://github.com/ICEI-PUC-Minas-PPLES-TI/plf-es-2025-2-tcci-0393100-dev-rafael-parreira.git>